UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

VICTOR GUILHERME CORTELETTE FERREIRA

GERAÇÃO PROCEDURAL DE MELODIAS COM AVALIAÇÃO PRÓPRIA, ANÁLISE DE DADOS LEVANTADOS PELAS AVALIAÇÕES

PATO BRANCO

2025

VICTOR GUILHERME CORTELETTE FERREIRA

GERAÇÃO PROCEDURAL DE MELODIAS COM AVALIAÇÃO PRÓPRIA, ANÁLISE DE DADOS LEVANTADOS PELAS AVALIAÇÕES

PROCEDURAL MELODY GENERATION WITH BUILT IN RATING, ANALYSIS OF DATA GENERATED BY RATINGS

Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Dr. Érick Oliveira Rodrigues

PATO BRANCO

2025

4.0 Internacional

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

RESUMO

Dentro do setor musical, é difícil obter informações concretas sobre a composição, devido a sua relatividade de sensações a cada ouvinte. Neste trabalho, visando contornar esta dificuldade, será explorado uso de geração procedural dentro de um sistema web para criar melodias automaticamente. Em virtude de coletar dados sobre as melodias geradas, haverá uma avaliação das melodias geradas pelo usuário, relacionada a qualidade e sensação disposta pela melodia no usuário, permitindo, posteriormente, a análise dos dados de avaliações e suas melodias correspondentes. Este trabalho se diferencia em relação a trabalhos relacionados por meio da análise do *feedback* do usuário acerca de melodias geradas, proporcionando uma visão de quais parâmetros de geração acarretam resultados melhores e de diferentes tipos, além do auxílio criativo oferecido pela geração em si.

Palavras-chave: melodia; geração procedural; música; técnicas de análise.

ABSTRACT

Within the musical area, it is hard to have concrete information regarding composition, due to its relativity of sensations to each listener. In this research paper, aiming to work around this difficulty, a web software employing procedural generation will be explored to create melodies automatically. In virtue of collecting data about the generated melodies, there will be evaluation of user generated melodies, allowing, posteriorly, the analysis of evaluation data and its corresponding melodies. This research differs in comparison to other related ones because of the analysis regarding the user feedback of generated melodies, providing a view of which generation parameters lead to better or certain types of results, in addition to the creative assistence provided by the generation itself.

Keywords: melody; procedural generation; music; analysis techniques.

LISTA DE FIGURAS

Figura 1 – Intervalos Musicais no piano 15
Figura 2 – Representações de notação de tempo 17
Figura 3 – Etapas de desenvolvimento do projeto 27
Figura 4 – Diagrama de Casos de Uso 35
Figura 5 – Diagrama de entidade Relacionamento 41
Figura 6 – Tela de Geração 43
Figura 7 – Tela de Avaliação 44
Figura 8 – Tela de Avaliação após avaliar 45
Figura 9 – Tela de Análise 45
Figura 10 – Tela de Análise filtrada 46
Figura 11 – Organização de pacotes do backend 47

LISTA DE TABELAS

Tabela 1 – Frequências das notas musicais da 1ª até a 8ª oitava 15
Tabela 2 – Escalas Maiores Naturais com Intervalos 16
Tabela 3 – Caso de uso - Gerar melodia 35
Tabela 4 – Caso de uso - Reproduzir melodia 36
Tabela 5 – Caso de uso - Avaliar melodia 37
Tabela 6 – Caso de uso - Baixar melodia como .mid 37
Tabela 7 – Caso de uso - Acesso a tela de análise 38
Tabela 8 – Caso de uso - Acesso a tela de análise 39
Tabela 9 – Estrutura da Tabela de Melodias 41
Tabela 10 – Estrutura da Tabela de Avaliações 42

LISTA DE QUADROS

Quadro 1 – Notas musicais e simbologias **14**
Quadro 2 – Ferramentas Utilizadas **26**

LISTAGEM DE CÓDIGOS FONTE

Listagem 1 – Pseudocódigo - Gerar melodia **28**

Listagem 2 – Pseudocódigo - Controller Geracao melodia **30**

Listagem 3 – Pseudocódigo - Classe Avaliação **31**

Listagem 4 – Pseudocódigo - Controller Geracao melodia **32**

Listagem 5 – Pseudocódigo - Repositório de Melodias e análises **33**

Listagem 6 – Pseudocódigo - Controller de Análises **34**

Listagem 7 – Classe GeradorMelodias - Método Principal **48**

Listagem 8 – Classe GeradorMelodias - métodos auxiliares **49**

Listagem 9 – Classe Melodia **50**

Listagem 10 – Classe GeracaoController **52**

Listagem 11 – Classe Avaliação **53**

Listagem 12 – Classe Avaliação Controller **54**

Listagem 13 – Classe MelodiaRepository **56**

Listagem 14 – Classe AnaliseController **58**

LISTA DE ABREVIATURAS E SIGLAS

Siglas

API	Application programming interface
BPM	Batimentos por minuto
DER	Diagrama Entidade Relacionamento
DTO	Data Transfer Object
HTTP	Hyper Text Transfer Protocol
MIDI	Musical Instrument Digital Interface
SQL	Structured Query Language
URL	Uniform Resource Locator

SUMÁRIO

1 INTRODUÇÃO 10

1.1 Objetivos 11

1.1.1 Objetivo geral 11

1.1.2 Objetivos específicos 11

1.2 Justificativa 11

1.3 Estrutura do trabalho 12

2 REFERENCIAL TEÓRICO 13

2.1 Composicão Musical e seus elementos 13

2.1.1 Notas e intervalos musicais 13

2.1.2 Escalas 15

2.1.3 Ritmo 16

2.1.4 Melodia 17

2.2 Geração Procedural 19

2.2.1 Geração baseada em Heurísticas 21

2.2.2 Geração procedural na música 22

2.3 Heurística 23

2.3.1 Heurística de Decisão 24

2.4 Técnicas de análise de dados 24

2.4.1 Análise quantitativa 24

3 MATERIAIS E MÉTODO 26

3.1 Materiais 26

3.2 Método 27

3.2.1 Estrutura do projeto 27

3.2.2 Implementação de geração de melodias 28

3.2.3 Implementação de avaliações 31

3.2.4 Implementação de análises 32

3.2.5 Diagrama de casos de uso 35

4 RESULTADOS 40

4.1 Escopo do sistema 40

4.2 Modelagem do sistema 40

4.2.1 Diagrama de Entidade Relacionamento Diagrama Entidade Relacionamento (DER) 41
4.2.2 Tabelas de Banco de Dados 41
4.3 Demonstração do sistema 43
4.4 Implementação do sistema 46
4.5 Comentário sobre funcionamento do sistema 58
4.6 Disponibilização do código fonte 59
5 CONCLUSÃO 60
REFERÊNCIAS 61

1 INTRODUÇÃO

A área de entretenimento e tecnologia têm evoluído lado a lado ao longo da história, e a música, como um dos formatos mais antigos de entretenimento, tem se integrado continuamente ao mundo moderno. Segundo Jones (1992), “sem a tecnologia, a música popular não existiria em sua presente forma”. Este relacionamento simbiótico entre tecnologia e música reflete a evolução contínua da indústria do entretenimento, impulsionada por avanços tecnológicos.

A integração da tecnologia no campo da música é evidenciada pela transformação que as ferramentas digitais proporcionaram. A tecnologia de software e computação tem desempenhado um papel fundamental nessa evolução. Manovich (2001) destaca que “a computação digital transformou todos os aspectos da cultura, desde a criação e distribuição até a recepção de conteúdos de entretenimento”. A produção musical, antes restrita a estúdios profissionais, tornou-se mais acessível através de softwares de edição de áudio, permitindo a criação e edição de faixas com uma precisão sem precedentes.

A geração procedural de melodias surge como uma possibilidade, fornecendo auxílio criativo ao construir melodias que podem ser integradas e adaptadas por compositores em suas próprias criações e oferecendo uma alternativa para a coleta de dados referentes a composição, requisitando ao compositor *feedback* acerca de gerações e posteriormente analisando os dados destes *feedbakcs*. Trabalhos recentes, como os de *Gobetti* (2022) e Agnol (2022), exploram técnicas de geração procedural para a criação de melodias, oferecendo novas possibilidades para a produção musical. No entanto, há um espaço significativo para aprofundar a pesquisa através da análise e estudo de dados, visando trazer informações mais concretas e objetivas para a área da música.

Para integrar efetivamente esses conceitos e expandir o alcance da pesquisa, a construção de um sistema web pode ser uma abordagem estratégica. A crescente presença da internet, que já alcança a vasta maioria do planeta, sublinha a afirmação de que “o mundo contemporâneo não vive mais sem internet”. Um sistema web poderia aproveitar esse alcance, visto que indica sistemas web como o mais comum e de maior afinidade ao público geral para coletar, analisar e compartilhar dados relacionados às melodias, oferecendo uma plataforma para avançar na pesquisa e prática musical de forma acessível e abrangente.

Este trabalho está estruturado de forma a abordar inicialmente os principais conceitos necessários para entender seu funcionamento, começando pelos relacionados à música, fornecendo uma base para a compreensão do tema. Em seguida, são apresentados os fundamentos da geração procedural, com uma breve introdução sobre os diferentes tipos existentes e um aprofundamento específico na geração com heurísticas e na geração dentro do contexto musical. Em seguida, finalizando a seção de definições teóricas, discute-se a aplicação de heurísticas de maneira mais aprofundada e técnicas de análise de dados, que se revelam fundamentais para interpretar os dados coletados pela geração e avaliação no sistema.

Após a fundamentação teórica, são abordados materiais e métodos utilizados no desenvolvimento do sistema, incluindo tecnologias utilizadas, fases do desenvolvimento, explicações acerca da implementação das principais funções implementadas e os casos de uso do sistema. Após isso, abordam-se o resultados gerados por esta pesquisa, iniciando por uma descrição do escopo do sistema, em seguida, uma explicação sobre o banco de dados utilizado e sua estrutura em detalhes, posteriormente, uma demonstração do sistema em sua camada visual, visualizando a experiência do usuário com o mesmo. Finalizando a seção de resultados, encontram-se detalhes sobre códigos utilizados para as principais funções do sistema e disponibilização de seu código fonte. Na conclusão deste trabalho, são retratadas principais ferramentas utilizadas, dificuldades durante seu desenvolvimento, denotações sobre os resultados obtidos, abordando a viabilidade do sistema e, por fim, possibilidades de futuras implementações a partir deste trabalho.

1.1 Objetivos

Nesta seção, são demonstrados os objetivos, divididos em geral e específicos. O objetivo geral é uma descrição abrangente da execução deste trabalho e os objetivos específicos descrevem as funcionalidades do sistema.

1.1.1 Objetivo geral

Encontrar, padrões entre o setor técnico da composição musical e os sentimentos experienciados pelo ouvinte. Dessa forma, compositores poderão se basear nos padrões para aprimorar futuras obras, observando quais parâmetros de teoria musical utilizados geração direcionam uma melodia a resultados melhores, ou com determinadas características.

1.1.2 Objetivos específicos

- Desenvolver um sistema de geração procedural de melodias com parâmetros;
- Obter avaliações de usuários quanto às melodias geradas no sistema;
- Fazer análises acerca dos dados levantados pelo sistema via avaliações.

1.2 Justificativa

A geração de melodias representa um recurso útil para compositores, oferecendo ideias iniciais que podem ser adaptadas e transformadas em peças musicais completas. Compositores frequentemente enfrentam desafios no processo criativo, como a dificuldade em iniciar uma

obra ou encontrar variações melódicas que complementem suas criações. Nesse sentido, um sistema automatizado que gera sugestões musicais pode servir como ponto de partida para novas ideias, tornando o fluxo criativo mais dinâmico e produtivo.

Neste contexto, este trabalho propõe a implementação de um gerador de melodias com um sistema de avaliação e análise das avaliações realizadas pelos usuários. Além de oferecer sugestões melódicas, o sistema visa permitir que os usuários avaliem as composições geradas, o que contribui para a coleta de dados relevantes sobre a recepção dessas melodias. Com base nessas avaliações, o sistema poderá identificar padrões e características associadas a diferentes percepções musicais.

A motivação principal deste projeto está no potencial de apoiar criadores musicais, tanto profissionais quanto amadores, ao ampliar o repertório de ideias melódicas disponíveis. Além de auxiliar diretamente na fase criativa, o sistema busca fornecer dados estatísticos que relacionem características da teoria musical com percepções emocionais das melodias geradas, como classificações entre tipos de sensação sentida pelo ouvinte.

Esses dados oferecem informações relevantes para compositores interessados em explorar padrões musicais específicos ou criar peças com determinadas intenções emocionais. Por exemplo, a análise revela como determinadas escalas ou ritmos influenciam a percepção emocional de uma melodia, fornecendo informações úteis para decisões composicionais mais fundamentadas.

Assim, o sistema contribue não apenas com sugestões melódicas iniciais, mas também com insights teóricos que possam orientar decisões composicionais de maneira fundamentada. Com o desenvolvimento desta aplicação, pretende-se unir tecnologia e arte, ampliando as possibilidades criativas e promovendo um novo entendimento das características musicais que influenciam a percepção do público, ao mesmo tempo em que oferece uma ferramenta prática para estimular a criação musical.

Buscando tornar a composição musical mais objetiva, com este software, usando a geração procedural, baseada em algoritmos, serão criadas melodias, que posteriormente serão analisadas pelo usuário, coletando dados que correlacionam a composição, ao sentimento experienciado pelo ouvinte, e o apreço do mesmo por esta melodia.

1.3 Estrutura do trabalho

Este trabalho é estruturado em capítulos, descritos a seguir. O Capítulo 1 contém a apresentação do trabalho, seus objetivos, e sua justificativa. No Capítulo 2 encontra-se a fundamentação teórica dos temas relacionados ao trabalho. O Capítulo 3 contém os materiais e métodos utilizados. O Capítulo 4 apresenta os resultados gerados pelo trabalho. Por fim, o Capítulo 5 apresenta a conclusão do trabalho.

2 REFERENCIAL TEÓRICO

Este capítulo contém a fundamentação teórica deste trabalho, seu conteúdo discorre sobre a composição musical e os elementos da teoria musical que a acompanham, geração procedural e os diferentes e seus diferentes estilos, e por fim apresenta as duas principais técnicas de análise de dados.

2.1 Composicão Musical e seus elementos

A composição musical é o processo de criação de uma peça musical e envolve uma combinação única de criatividade, habilidade técnica e conhecimento teórico. Segundo Ferraz Ferraz (2013), a composição musical é um campo interdisciplinar que articula filosofia, biologia, física, literatura, linguística, tecnologias eletrônicas e digitais, além da técnica composicional. É por meio da composição que os compositores podem expressar emoções, contar histórias ou transmitir mensagens específicas. A compreensão dos elementos da teoria musical é essencial para a prática da composição, pois fornece as ferramentas e a linguagem necessária para a expressão musical Barros e Ansay (2015).

Além disso, a composição musical na universidade brasileira tem sido objeto de pesquisa e debate, destacando-se a importância da interação entre compositores, estudantes e pesquisadores na área de teoria da composição Ferraz (2013). Essa interação não só enriquece o processo criativo, mas também contribui para a formação de uma base teórica sólida que apoia a prática composicional.

A prática da composição musical também pode ser vista como uma ferramenta terapêutica, como demonstrado por Barros e Ansay Barros e Ansay (2015), que exploram a experiência da composição na musicoterapia. Eles destacam que o processo de composição pode desenvolver a criatividade, o poder de argumentação e a liberdade de expressão dos participantes. Assim, a composição musical não só serve como um meio de expressão artística, mas também como uma prática que pode ter benefícios significativos em contextos terapêuticos e educacionais.

2.1.1 Notas e intervalos musicais

Notas musicais são ondas que se propagam pelo ar com uma determinada frequência em hertz (Hz), denominada altura do som. Segundo Moraes (2015), o que popularmente é chamado de altura de um som é, na verdade, a intensidade sonora. Se colocarmos, por exemplo, a mesma música em dois aparelhos de som com potências diferentes, a “altura máxima” será a mesma; o que irá mudar é a intensidade. Em Física, a amplitude de um som refere-se ao quão agudo esse som é. Quanto mais alto o som, maior sua frequência; logo, mais agudo ele será. Da

mesma forma, sons mais baixos são chamados de graves. De acordo com Jeremy (2021), no sistema musical seguido no Ocidente, denominado dodecafonismo e desenvolvido pelo músico austríaco Arnold Schoenberg no início do século XX, há sete notas musicais: Dó, Ré, Mi, Fá, Sol, Lá e Si. Além dessas sete, existem as notas musicais denominadas acidentes, sustenidos ou bemóis, que ocorrem quando se definem espaços menores entre cinco das sete notas musicais. Cada acidental ascendente (sustenido) tem seu equivalente descendente (bemol), como pode ser observado no Quadro 1.

Quadro 1 – Notas musicais e simbologias

Nota Musical	**Simbologia**
Dó	C
Dó sustenido/Ré bemol	C# / Db
Ré	D
Ré sustenido/Mi bemol	D# / Eb
Mi	E
Fá	F
Fá sustenido/Sol bemol	F# / Gb
Sol	G
Sol sustenido/Lá bemol	G# / Ab
Lá	A
Lá sustenido/Si bemol	A# / Bb
Si	B

Fonte: Autoria própria (2024).

Essas notas podem ser alteradas de acordo com as tonalidades, que seriam as diferenças de intervalo entre cada uma. Após percorrer a sequência das sete notas, ocorre uma mudança de oitava, na qual as notas musicais reiniciam na ordem citada. Caso sejam percorridas decrescentemente, serão notas mais graves (de menor frequência); se percorridas ascendentemente, serão notas mais agudas (de maior frequência). Ou seja, é possível, por exemplo, ter duas notas Dó que soam de maneira diferente ao ouvido, pois têm frequências distintas, mas são representadas pela mesma nota musical. Na Tabela 1, podem ser observadas as frequências de cada nota e acidente nas primeiras 8 oitavas.

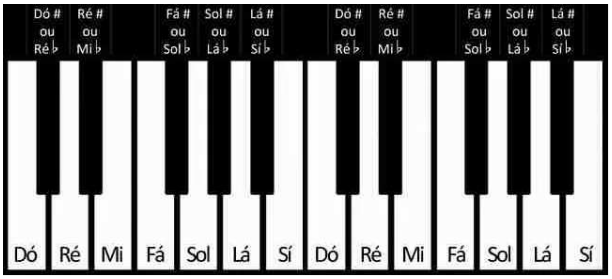
Tabela 1 – Frequências das notas musicais da 1ª até a 8ª oitava

Nota	1ª Oitava	2ª Oitava	3ª Oitava	4ª Oitava	5ª Oitava	6ª Oitava	7ª Oitava	8ª Oitava
C	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01
C#	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92
D	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.63
D#	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03
E	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04
F	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65
F#	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91
G	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93
G#	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88
A	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00
A#	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62
B	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13

Fonte: Autoria própria (2024).

Um intervalo musical refere-se à distância que separa duas notas musicais e estabelece uma relação entre elas. Quando são tocadas duas notas ao mesmo tempo ou em sequência, há uma diferença na frequência dos sons emitidos. Essa diferença faz com que nosso ouvido as perceba como distintas, e é denominado intervalo musical (Schmidt, 2018). Diz-se que a menor distância que separa dois sons é de um semitom, como a distância entre C (Dó) e C# (Dó sustenido), por exemplo. Um tom é a soma de dois semitons, como a distância entre C (Dó) e D (Ré). Para exemplificar, no piano ou teclado, o semitom seria a distância entre uma tecla branca e uma tecla preta, ou a distância entre teclas brancas que não têm uma tecla preta entre elas, como entre Si e Dó e entre Mi e Fá (Silva, 2025). Na Figura 1, são observados os intervalos citados no piano.

Figura 1 – Intervalos Musicais no piano



Fonte: Opus Ensino Musical (2018).

2.1.2 Escalas

Segundo a (Britannica, 2024) uma escala musical é uma sequência ordenada de notas que divide uma oitava. O tipo de escala é determinado pelos intervalos entre cada nota da sequência. Ao seguir esses intervalos a partir de cada nota musical, obtém-se o tipo específico de escala para essa nota. A escala maior, por exemplo, é conhecida por produzir sons alegres

e mais agitados. Essa escala é composta por uma sequência de intervalos: tom, tom, semitom, tom, tom, tom e semitom novamente.

Tomando como exemplo a escala de Dó (C) maior. Nesta escala, a nota Dó dá o nome à escala. A sequência é composta por Ré como a 2ª nota (a uma distância de 1 tom de Dó), Mi como a 3ª (a 1 tom de Ré), Fá como a 4ª (a meio tom de Mi), Sol como a 5ª (a 1 tom de Fá), Lá como a 6ª (a 1 tom de Sol) e Si como a 7ª (a 1 tom de Lá). Finalmente, a escala se completa com mais um intervalo de semitom, retornando à nota Dó, uma oitava acima.

A relação entre a música ocidental e matemática é bem documentada. Pitágoras, por exemplo, descobriu que as frequências sonoras podem ser expressas em proporções matemáticas simples, como razões entre números inteiros. Essas proporções são fundamentais para a formação das escalas musicais e para a harmonia musical (Pereira; Madruga, 2020). Além disso, a modelagem matemática pode ser usada para representar escalas musicais, facilitando o ensino e a aprendizagem de conceitos matemáticos na educação básica (Sodré, 2015). A Tabela 2 ilustra as escalas maiores de cada nota não acidental (ou seja, sem bemóis ou sustenidos).

Tabela 2 – Escalas Maiores Naturais com Intervalos

Nota	1º	2º	3º	4º	5º	6º	7º	8º
Intervalo	-	T	T	S	T	T	T	S
Dó	Dó	Ré	Mi	Fá	Sol	Lá	Si	Dó
Ré	Ré	Mi	Fá#	Sol	Lá	Si	Dó#	Ré
Mi	Mi	Fá#	Sol#	Lá	Si	Dó#	Ré#	Mi
Fá	Fá	Sol	Lá	Si♭	Dó	Ré	Mi	Fá
Sol	Sol	Lá	Si	Dó	Ré	Mi	Fá#	Sol
Lá	Lá	Si	Dó#	Ré	Mi	Fá#	Sol#	Lá
Si	Si	Dó#	Ré#	Mi	Fá#	Sol#	Lá#	Si

Fonte: Autoria própria (2024).

2.1.3 Ritmo

O ritmo é um elemento fundamental na estrutura temporal da música, responsável por organizar os sons e silêncios em uma sequência regular. Ele serve como a base sobre a qual a melodia e a harmonia são construídas, determinando o movimento e a pulsação da música. Segundo a *Britannica* (2024), o ritmo pode ser descrito como a organização de tempos fortes e fracos em um padrão repetitivo, cuja regularidade pode variar

Para compreender a duração das notas e a organização do ritmo na música, é essencial entender o conceito de compasso. Um compasso musical é definido como uma divisão da música em intervalos de tempo iguais, com o objetivo de organizar a estrutura e facilitar a orientação para o leitor. Esse intervalo de tempo é representado por barras verticais na partitura. Por exemplo, no compasso 4/4, cada compasso contém quatro tempos, e cada tempo é equivalente a uma semínima (Descomplicando a Música, 2014). Na notação rítmica, a duração das notas

é representada por diferentes figuras musicais. A semínima possui a duração de um tempo em um compasso 4/4. A colcheia, por sua vez, tem a duração de metade de uma semínima, sendo comum encontrar frases rítmicas que combinam semínimas e colcheias para criar padrões variados. A mínima equivale a dois tempos em um compasso 4/4, enquanto a semibreve dura quatro tempos (Academia Musical, 2023). Além disso, a pausa de semínima representa um silêncio de um tempo, interrompendo a sequência de notas. Na Figura 2, observam-se as representações, durações e nomenclaturas das figuras de tempo musical.

Figura 2 – Representações de notação de tempo

Nome	Figuras de Som	Figuras de Silêncio	Duração
Semibreve			4 tempos
Mínima			2 tempos
Semínima			1 tempo
Colcheia			½ tempo
Semicolcheia			¼ tempo
Fusa			⅛ tempo
Semifusa			1/16 tempo

Fonte: Martins (2018).

Em música, a frase rítmica pode ser entendida como uma unidade que organiza a duração das notas e os silêncios de maneira que resulta em uma estrutura musical clara. De acordo com Schoenberg (2011), a frase musical é como uma unidade que pode ser "cantada em um só fôlego", com seu término sugerindo uma pontuação, como uma vírgula. Por exemplo, uma frase rítmica simples pode combinar semínimas e colcheias em um compasso 4/4, criando um padrão de ritmo que serve de base para a melodia. O ritmo pode ser simples, como em uma marcha com batidas regulares, ou mais complexo, como em ritmos sincopados que aparecem em várias tradições musicais (Titon, 2014). O ritmo influencia diretamente a sensação de movimento e o caráter da música, ajudando a criar padrões que são percebidos como agradáveis ou intrigantes (Dowling; Harwood, 1986).

2.1.4 Melodia

A melodia é uma sequência de notas musicais que são percebidas como uma linha coesa e significativa, constituindo-se como um elemento essencial na música, responsável por capturar a atenção do ouvinte e ser facilmente memorizada. Segundo a Enciclopédia (Britannica, 2024), a melodia é formada por uma série de notas organizadas de acordo com uma

escala musical. A escolha das notas e a sua disposição temporal determinam tanto a qualidade quanto o impacto emocional da melodia.

As melodias são estruturadas em frases que criam uma sensação de continuidade e desenvolvimento, e as notas que as compõem são, em geral, selecionadas a partir de uma escala musical específica. Além disso, o ritmo desempenha um papel crucial ao influenciar a duração das notas e o espaçamento entre elas, contribuindo para a expressividade da melodia. De acordo com Piston e DeVoto (1987), a melodia é frequentemente o elemento mais memorável de uma composição, sendo responsável por expressar a ideia musical principal.

A importância da melodia na música é amplamente reconhecida na literatura acadêmica. Segundo *Silva* e *Oliveira* (2019), a melodia é fundamental para a construção da identidade musical de uma obra, sendo capaz de evocar emoções e criar uma conexão profunda com o ouvinte. Souza (2020) observa que a melodia, quando bem estruturada, tem um impacto significativo no reconhecimento e retenção de uma peça musical. Um exemplo claro dessa afirmação pode ser observado na música infantil "Twinkle, Twinkle, Little Star"de Taylor e Mozart (1806), reconhecida e lembrada em diversas culturas ao redor do mundo tornando-a uma das canções mais cantadas em escolas de educação infantil nos Estados Unidos, conforme indicado pela Composers e ASCAP (2003). A melodia simples e repetitiva torna-a memorável. Com intervalos pequenos e um padrão melódico repetido, essa canção se torna facilmente reconhecível para os ouvintes, independentemente de sua idade. A repetição e a simplicidade de sua estrutura facilitam tanto o aprendizado quanto o reconhecimento imediato, evidenciando como a melodia, ao ser bem estruturada, pode melhorar a retenção de uma música, como enfatizado por Souza (2020).

A melodia desempenha um papel crucial na diferenciação de gêneros musicais. Conforme apontado por Lima e Ferreira (2021), diferentes estilos musicais utilizam estruturas melódicas distintas, o que contribui para a diversidade e riqueza da música. A música clássica tende a empregar melodias complexas e elaboradas, como as de Beethoven, que utilizam modulações para tonalidades distantes e transformações temáticas, características que a distinguem de estilos mais acessíveis. Por exemplo, em sua Sinfonia nº 5, Beethoven usa modulações, o processo de transição entre tonalidades diferentes dentro de uma obra Dobbert (2020), para ampliar a paleta harmônica e criar contrastes emocionais profundos. Além disso, a obra faz uso de transformações temáticas, onde um motivo simples é repetido e modificado ao longo da peça, assumindo diversas formas e características ao longo do tempo.

O motivo em questão é tema de quatro notas que abre a Sinfonia nº 5 de Beethoven. Esse motivo, composto por uma sequência curta de notas composta de um intervalo de terça menor e dois de semitom, serve como a base para o desenvolvimento temático. Embora o motivo em si seja curto e direto, ele é manipulado e expandido de várias maneiras ao longo da sinfonia, sendo transformado por meio de mudanças rítmicas, harmônicas e melódicas.

Por outro lado, artistas da música popular, como Michael Jackson e Adele, frequentemente utilizam melodias mais simples e repetitivas. As melodias de seus sucessos, como “Billie Jean” (Jackson (1982)) ou “Someone Like You” (Adele (2011)), são baseadas em estruturas me-

lódicas previsíveis. Isso ocorre porque essas músicas frequentemente utilizam intervalos menores, como segundos e terceiros, que são intervalos mais próximos entre as notas e, portanto, mais fáceis de prever e memorizar. Além disso, há pouca ou nenhuma modulação, ou seja, a tonalidade permanece estável ao longo da música, o que diminui a surpresa harmônica.

Outro aspecto relevante é a relação entre melodia e harmonia. Segundo estudos de Almeida e Santos (2022), a interação entre esses elementos é essencial para a criação de uma peça musical coesa e harmoniosa. A melodia, definida como uma sequência de notas que são percebidas como uma unidade e reconhecíveis como uma linha musical fluida Piston (1962), pode ser enriquecida pela harmonia. A harmonia, por sua vez, envolve a combinação de notas ou acordes que acompanham a melodia e fornecem contexto sonoro, criando uma base emocional que amplia o impacto da composição Almeida e Santos (2022). A harmonia oferece o suporte necessário para que a melodia se desenvolva de maneira completa e expressiva, enquanto a melodia proporciona a direção e a identidade da peça musical.

Por fim, a evolução tecnológica tem permitido novas formas de criação e manipulação de melodias. De acordo com pesquisa de Costa e Pereira (2023), o uso de softwares de composição musical tem facilitado a experimentação e inovação melódica, permitindo aos compositores explorar novas possibilidades e expandir os limites da criatividade musical. Costa e Pereira (2023) destacam que as ferramentas digitais oferecem aos compositores uma maior liberdade para explorar estruturas melódicas mais complexas e interagir com diferentes elementos musicais de maneiras que antes seriam difíceis ou impossíveis sem o uso de tecnologias computacionais.

Costa e Pereira (2023) explicam que a possibilidade de manipulação de áudio e a criação de sequências melódicas com softwares avançados permitem que os compositores experimentem variações melódicas instantaneamente e realizem ajustes em tempo real.

2.2 Geração Procedural

A geração de conteúdo procedural, conforme descrita por Togelius *et al.* (2011), refere-se à criação algorítmica de conteúdo com mínima ou indireta intervenção do usuário. No universo dos jogos, essas técnicas têm ganhado destaque por permitirem a criação de experiências diversas sem a necessidade de intervenção manual do desenvolvedor em cada cenário específico. Essa abordagem possibilita, por exemplo, a geração de mapas variados, combinações distintas de itens e diferentes arranjos de inimigos a cada sessão de jogo.

Exemplos notáveis incluem jogos como Spore Maxis (2008) e The Binding of Isaac McMillen (2011), sendo este último particularmente inovador ao utilizar a geração procedural para criar novas configurações de salas e itens dentro do jogo, proporcionando uma experiência sempre renovada para o jogador.

Dentre as várias técnicas de geração procedural, destaca-se o uso de L-Systems, um modelo algorítmico proposto por Lindenmayer (1968) para descrever o crescimento de plantas

e organismos. Essa técnica é amplamente utilizada na criação de vegetação e estruturas auto-similares em ambientes virtuais.

Em jogos como Minecraft Mojang Studios (2011), L-Systems permitem a geração eficiente de árvores e outros elementos naturais, criando uma sensação de mundo vivo e em constante mudança. Os L-Systems operam por meio de regras de substituição aplicadas a símbolos, que geram padrões complexos em sucessivas iterações. Outro tipo de geração procedural é o uso de fractais, que são figuras geométricas que exibem auto-similaridade, ou seja, seus padrões se repetem em diferentes escalas. Mandelbrot (1982) foi pioneiro no estudo de fractais, e sua aplicação na geração de terrenos e paisagens naturais tem se mostrado extremamente eficaz em jogos e simulações. Técnicas como o Perlin (1985), que é baseado em fractais, são comumente usadas para criar terrenos dinâmicos e realistas em jogos, como Spore (Maxis, 2008). A utilização de fractais em jogos permite a criação de paisagens vastas, com variações naturais e detalhadas, como montanhas, rios e florestas, sem a necessidade de intervenção manual em cada elemento.

Ao adotar algoritmos para a produção dinâmica de conteúdo, os desenvolvedores economizam tempo e recursos, criando um único algoritmo parametrizável que pode ser reutilizado em múltiplos contextos. Essa economia permite que o foco seja direcionado para aspectos mais criativos e amplos do design do jogo, enriquecendo a experiência do jogador com novos desafios e ambientes a cada partida. Além de ampliar as possibilidades de design, essas técnicas contribuem para aumentar a longevidade dos jogos, mantendo o interesse dos jogadores por mais tempo.

Silva, Marson e Cassol (2016) destacam que a geração procedural não se trata apenas de automatizar a criação de conteúdo, mas também de garantir que os elementos gerados sejam coerentes e expressivos dentro do contexto especificado. Para os autores, a expressividade na geração procedural envolve definir restrições e parâmetros que assegurem que o conteúdo produzido esteja alinhado com a experiência desejada Além disso, eles ressaltam a importância da adaptação dinâmica, permitindo que os algoritmos ajustem o conteúdo gerado com base no comportamento e preferências do usuário.

Além dos jogos, a geração procedural se estende a outros contextos, como simulações e ambientes virtuais. Em simulações de treinamento, por exemplo, pode criar cenários variados e adaptativos que preparam os usuários para situações inesperadas. Nos ambientes virtuais, possibilita a construção de mundos ricos e dinâmicos, que se ajustam às interações dos usuários, oferecendo experiências mais personalizadas e imersivas. Dessa forma, a geração procedural se configura como uma ferramenta poderosa e versátil, com aplicações que transcendem o campo dos jogos eletrônicos, demonstrando seu potencial em diversas áreas tecnológicas.

Em síntese, a geração procedural se destaca como uma ferramenta poderosa para criar conteúdo dinâmico e diversificado, sendo aplicada amplamente no desenvolvimento de jogos e simulações. Essa abordagem, além de otimizar tempo e recursos, também contribui para a criação de experiências imersivas e adaptativas. Contudo, embora a geração procedural seja eficaz

na criação de conteúdos vastos e complexos, ela pode ser limitada pela necessidade de parâmetros rígidos e pela falta de adaptação a situações mais específicas ou inesperadas. Nesse cenário, a geração heurística surge como uma alternativa interessante, oferecendo métodos baseados em regras para otimizar ainda mais o processo de criação.

2.2.1 Geração baseada em Heurísticas

A geração de conteúdo baseada em heurísticas, formato de geração usado neste trabalho, utiliza um conjunto de regras predefinidas para estruturar o processo criativo. Essa abordagem se destaca por sua simplicidade e previsibilidade, tornando-se uma solução eficiente para cenários que exigem resultados consistentes. Sua principal vantagem reside na facilidade de implementação, permitindo a aplicação direta de padrões estabelecidos no processo de geração Smith, Cha e Whitehead (2010). A geração heurística é, por exemplo, particularmente útil em jogos de tabuleiro digitais, como Chess e Go, onde as regras do jogo precisam ser rigorosamente seguidas para garantir a coerência do ambiente e das interações Yannakakis e Togelius (2018).

No entanto, essa previsibilidade também pode ser uma limitação. Quando as heurísticas são excessivamente restritivas, a diversidade do conteúdo gerado pode ser comprometida, reduzindo a capacidade de inovação. Esse método é especialmente adequado para contextos que demandam uma estrutura fixa e a manutenção da coesão entre diferentes instâncias geradas. Além disso, a geração heurística é amplamente empregada em situações onde a integridade estrutural e a conformidade com padrões são essenciais, como em jogos de tabuleiro digitais ou simulações com design previsível e estruturado. Nesses casos, garante-se que o conteúdo gerado atenda a requisitos específicos de funcionalidade e coesão.

Togelius *et al.* (2011) argumentam que, embora a geração heurística ofereça previsibilidade e controle, sua rigidez pode limitar a inovação e a adaptabilidade do conteúdo gerado. Para mitigar esse problema, os autores sugerem a combinação de heurísticas com técnicas baseadas em busca e aprendizado de máquina, permitindo que o sistema refine automaticamente suas regras com base em exemplos ou *feedback* iterativo. Esse tipo de abordagem híbrida tem sido utilizado em contextos que requerem tanto estrutura quanto variabilidade, como em sistemas de design procedural que mesclam regras fixas com variações algorítmicas para ampliar a diversidade dos cenários gerados. Além disso, a geração baseada em heurísticas tem sido aplicada além do design de jogos, estendendo-se a outros domínios criativos. No campo da música, por exemplo, técnicas algorítmicas têm permitido a composição dinâmica e a adaptação sonora em tempo real, como ilustrado no trabalho de Novack *et al.* (2011), que exploram a criação de partituras musicais utilizando heurísticas para gerar melodias e harmonias que se ajustam a padrões preestabelecidos, aplicando a geração por heurísticas a um contexto similar ao deste trabalho. Isso mostra que a geração por heurísticas, embora frequentemente associ-

ada à criação de jogos e ambientes virtuais, também pode ser aplicada em diferentes campos criativos.

Dessa forma, a geração baseada em heurísticas se mostra uma ferramenta valiosa para a criação de conteúdo estruturado e previsível, especialmente quando integrada a abordagens mais flexíveis, como algoritmos evolutivos e aprendizado de máquina. No entanto, sua aplicação não se limita apenas ao design de níveis e ambientes em jogos. A proceduralidade tem sido explorada em diversas áreas criativas, incluindo a música, onde técnicas algorítmicas permitem a composição dinâmica e a adaptação sonora em tempo real.

2.2.2 Geração procedural na música

A produção musical procedural utiliza algoritmos para criar melodias e composições de forma autônoma ou com intervenção mínima do usuário. Gramáticas formais e heurísticas harmônicas são abordagens comuns, nas quais algoritmos aplicam modelos da teoria musical para gerar melodias coerentes (Pacheco *et al.*, 2021). Segundo *Pacheco et al.* (2021), essas técnicas permitem a criação de músicas que seguem padrões harmônicos estabelecidos, garantindo a coerência musical.

A geração baseada em algoritmos evolutivos cria e refina melodias através de processos de seleção e mutação, permitindo composições únicas e incorporando as preferências do usuário (Silva; Oliveira, 2022). *Silva* e *Oliveira* (2022) destacam que, embora essa abordagem permita a personalização das composições, a complexidade computacional é uma desvantagem significativa.

O aprendizado de máquina também é aplicado, com modelos treinados em grandes bancos de dados musicais para gerar novas músicas que capturem estilos diferentes (Briot; Hadjeres; Pachet, 2020). A vantagem é a originalidade e adaptabilidade, mas isso depende da qualidade dos dados de treinamento (Briot; Hadjeres; Pachet, 2020). De acordo com *Briot*, *Hadjeres* e *Pachet* (2020), a utilização de redes neurais profundas permite a criação de composições que imitam estilos musicais específicos, proporcionando uma grande diversidade de resultados.

Além disso, a aplicação de técnicas de aprendizado profundo tem mostrado resultados promissores na geração de música. Redes neurais recorrentes e redes generativas adversariais são frequentemente utilizadas para criar composições que variam de música clássica a jazz (Souza; Almeida, 2023). *Souza* e Almeida (2023) afirmam que essas técnicas não apenas geram músicas originais, mas também podem ser ajustadas para incorporar preferências estilísticas específicas dos usuários.

Outro aspecto relevante é a utilização de algoritmos de inteligência artificial para a análise e síntese de áudio, permitindo a criação de novas texturas sonoras e efeitos musicais (Ferreira; Costa, 2023). *Ferreira* e *Costa* (2023) ressaltam que essas técnicas ampliam as possibilidades criativas dos compositores, oferecendo ferramentas inovadoras para a produção musical.

Como exemplos de geração de melodias, o trabalho de *Gobetti* (2022) foca na construção de um sistema web que permite a geração de melodias com base em parâmetros definidos pelo usuário, com foco na coleta de *feedback* dos usuários sobre a qualidade das melodias geradas. A ênfase principal deste trabalho está na experiência do usuário e na avaliação subjetiva das melodias, destacando-se a inclusão de um sistema de *feedback* que possibilita a consulta às dez melodias mais bem avaliadas. Esta abordagem sugere um interesse em compreender a percepção do usuário em relação à qualidade estética das melodias, o que pode ser valioso para ajustes e melhorias no algoritmo de geração. Este trabalho se diferencia do trabalho de (Gobetti, 2022) ao guardar não apenas uma avaliação de qualidade das melodias geradas, mas também uma avaliação acerca da sensação sentida pelo mesmo ao ouvir a geração.Ademais, é possível analisar os parâmetros utilizados em gerações, filtrando-os pelas sensações e qualidade avaliadas.

O trabalho de Agnol (2022) explora a geração procedural de melodias utilizando técnicas de aprendizado de máquina, com destaque para o uso de Redes Neurais. Este trabalho se concentra na capacidade das Redes Neurais de identificar tendências e aplicar heurísticas musicais complexas para a composição de melodias em diferentes gêneros musicais. Este trabalho se diferencia do trabalho de Agnol (2022) tanto pelo estilo de geração procedural, visto que este trabalho implementa uma geração baseada em heurísticas, em contraponto ao uso de aprendizado de máquina implementado pelo mesmo, quanto pela implementação de um sistema de avaliações e análise de dados para as melodias geradas.

2.3 Heurística

Heurísticas são estratégias ou métodos utilizados para encontrar soluções aproximadas para problemas complexos ou mal definidos de forma rápida e eficiente. Ao invés de buscar a solução perfeita, as heurísticas se concentram em encontrar uma solução “boa o suficiente” que pode ser obtida mais rapidamente do que com métodos exatos. Elas são amplamente utilizadas em diversos campos, como inteligência artificial, otimização, e ciência da computação, para enfrentar desafios onde soluções rápidas são necessárias ou onde o custo de computação de uma solução exata é muito alto (Kahneman; Tversky, 1974).

As heurísticas não são apenas ferramentas teóricas, mas também têm aplicações práticas em diversas áreas. Na ciência da computação, por exemplo, algoritmos heurísticos são utilizados para resolver problemas de otimização onde métodos exatos seriam computacionalmente inviáveis. Esses algoritmos são projetados para encontrar soluções satisfatórias em um tempo razoável, o que é essencial em aplicações como roteamento de redes, planejamento de produção e alocação de recursos (Nunes; Santos, 2021). A utilização de heurísticas permite que sistemas de inteligência artificial e aprendizado de máquina façam previsões e tomem decisões de forma eficiente, mesmo quando enfrentam grandes volumes de dados e complexidade computacional.

2.3.1 Heurística de Decisão

Heurísticas de decisão referem-se a heurísticas simplificadas que ajudam os indivíduos a tomarem decisões rápidas e eficazes sem a necessidade de uma análise completa de todas as informações disponíveis. Elas são frequentemente baseadas em experiências passadas e intuições, e são usadas para reduzir a complexidade cognitiva em situações de incerteza.

Tonetto et al. (2006). Apesar de poderem levar a erros ou vieses, as heurísticas de decisão são fundamentais em contextos onde é necessário tomar decisões rápidas, como em negócios, saúde e psicologia comportamental (Moraes; Tabak, 2018). Essas heurísticas permitem que as pessoas naveguem em ambientes complexos de maneira mais eficiente, embora possam resultar em soluções subótimas.

Segundo (Tonetto *et al.*, 2006), as heurísticas mais comuns incluem a heurística de ancoragem, onde os indivíduos baseiam suas decisões em um ponto de referência inicial, e a heurística de disponibilidade, onde a facilidade com que exemplos vêm à mente influencia a percepção de probabilidade. Essas heurísticas são particularmente úteis em situações de incerteza, onde a informação completa não está disponível.

Em um estudo sobre a aplicação de heurísticas na área da saúde, (Silva; Almeida, 2020) demonstraram que profissionais de saúde frequentemente utilizam heurísticas para tomar decisões rápidas em situações de emergência. No entanto, eles também ressaltam a necessidade de treinamento adequado para minimizar os vieses e melhorar a precisão das decisões.

Ademais, no contexto deste trabalho, as heurísticas de decisão na geração procedural permitem a geração de conteúdos em formato padronizado devido a forma de parametrização utilizada, relacionada ao formato heurístico de geração. Em virtude desta padronização, é possível aplicar técnicas de análise de dados acerca da parametrização utilizada na geração, possibilitando a extração de informações estatísticas acerca do conteúdo gerado.

2.4 Técnicas de análise de dados

A análise de dados constitui um elemento essencial do processo de investigação e tomada de decisão em diversas áreas do conhecimento, desde as ciências sociais e humanas até às ciências exatas e aplicadas. Neste trabalho, técnicas de análise de dados se mostram essenciais para a coleta de informações baseadas nas melodias avaliadas.

2.4.1 Análise quantitativa

A análise de dados quantitativos, utilizada neste trabalho, envolve a coleta e exame de dados numéricos com o objetivo de testar hipóteses, identificar padrões, fazer previsões e tomar

decisões com base em evidências concretas. Esse tipo de análise é amplamente empregado por sua capacidade de oferecer insights objetivos e reprodutíveis.

Conforme destacado por *Hair et al.* (2014), a análise quantitativa permite a descrição, conclusão e generalização dos dados, sendo uma poderosa ferramenta para validar hipóteses e teorias. Técnicas fundamentais incluem a estatística descritiva, que resume as características principais de um conjunto de dados por meio de medidas como média, mediana, moda, desvio padrão e distribuição de frequência (Fávero *et al.*, 2009). Essas métricas são essenciais para identificar tendências centrais, variabilidade e padrões de distribuição dos dados.

Adicionalmente, técnicas mais avançadas, como análise de regressão, testes de significância e modelos de séries temporais, permitem a investigação de relações complexas entre variáveis e a previsão de fenômenos futuros. A análise de correlação, por exemplo, mede a intensidade das associações entre variáveis, enquanto modelos preditivos ajudam a projetar cenários com base em dados históricos. Um exemplo recente da aplicação da análise quantitativa é o estudo de (Araújo, 2020), que analisou a influência do TikTok na indústria musical por meio de métricas como visualizações e engajamento, demonstrando como plataformas digitais podem impactar o sucesso de lançamentos musicais.

Uma característica central da análise quantitativa é sua objetividade, alcançada por meio do uso sistemático de números e estatísticas, o que reduz a subjetividade do pesquisador (Hair *et al.*, 2014). A reprodutibilidade é outro benefício, uma vez que os métodos podem ser replicados e verificados por outros pesquisadores. Além disso, a generalização dos resultados obtidos a partir de amostras permite aplicar as conclusões a conjuntos de dados maiores, desde que os pressupostos amostrais sejam devidamente atendidos.

Entretanto, a análise quantitativa apresenta limitações que precisam ser consideradas. A necessidade de amostras grandes para garantir resultados confiáveis pode representar um desafio prático. Além disso, a abordagem pode não capturar nuances contextuais importantes, limitando sua aplicabilidade em situações onde variáveis qualitativas desempenham um papel central.

No contexto deste trabalho, a análise quantitativa é fundamental para avaliar as avaliações de melodias geradas proceduralmente. As técnicas utilizadas englobam estatísticas descritivas e análises de correlação, com o objetivo de identificar quais configurações de parâmetros apresentam avaliações com notas mais altas e quais são relacionadas a cada sensação que pode ser escolhida pelos usuários para descrever a melodia gerada.

3 MATERIAIS E MÉTODO

A seguir estão representados os materiais e métodos utilizados no desenvolvimento desde trabalho.

3.1 Materiais

No Quadro 2, estão listadas as ferramentas e tecnologias que serão usadas no desenvolvimento do trabalho.

Quadro 2 – Ferramentas Utilizadas

Ferramenta/Tecnologia	Versão	Finalidade
Java	22	Linguagem de Programação
Spring Boot	3.2.4	Framework para uso da linguagem java em sistema web
IntelliJ	2024.1.5	IDE para desenvolvimento Java
Visual Studio Code	1.92	IDE para desenvolvimento front-end
Lombok	1.18	Biblioteca para sistemas Java
JFugue	5.0	Biblioteca para trabalho com música em Java
HTML5	5.2	Linguagem de marcação
JavaScript	5.1	Linguagem para desenvolvimento front-end
CSS	12.1	Linguagem para estilização das páginas
React	18.3.1	Biblioteca para javascript front-end
PrimeReact	10.9.1	Biblioteca de componentes para React
MySQL	9.0.1	Banco de dados
MySQL Workbench	8.0.38	Ferramenta de manipulação de banco de dados
Postman	11.2.0	Ferramenta de teste de requisições HTTP/HTTPS
Visual Paradigm	17.2	Ferramenta para criação de diagramas

Fonte: Autoria própria (2025).

No desenvolvimento do sistema web, foi utilizado Java como a linguagem de programação principal, com o Spring Boot servindo como *framework* para auxiliar no desenvolvimento da aplicação em formato web. Para o desenvolvimento em Java, será utilizado o IntelliJ, enquanto o Visual Studio Code será empregado para trabalhar com linguagens HTML5, CSS e Javascript, na seção de *front-end* do trabalho, que, com auxílio da biblioteca React, estrutura e estiliza as páginas web. A biblioteca Lombok será utilizada para simplificação do código Java, e o JFugue permite o trabalho com música e seus elementos dentro do ambiente Java. O sistema de banco de dados utilizado será MySQL, e a ferramenta MySQL Workbench será utilizada para a manipulação e gestão do banco de dados.

3.2 Método

No desenvolvimento desse trabalho, serão aplicados métodos de geração procedural com heurísticas para criar melodias a partir de parâmetros fornecidos pelo usuário, coletar avaliações acerca das melodias geradas e por fim, serão feitas análises a partir dos dados coletados. Estas heurísticas consistem em parametrizar a geração, delegando ao usuário a escolha para instrumento, escala musical, bpm e a quantidade máxima de oitavas permitidas para a geração, e mantendo fixos os parâmetros para a oitava mais baixa permitida para as notas, duração máxima permitida da melodia e o máximo de repetições permitidas para cada nota da escala. A coleta de avaliações é feita por meio de uma nota de 0 a 10 enviada pelo usuário, e a definição de uma sensação que o usuário usaria para melhor descrever o que sentiu ao ouvir a melodia. Por fim, a análise das avaliações é feita filtrando a quantidade de melodias geradas definidas dentro de faixas de nota altas (7-10) para cada opção de parâmetro, ou, filtrando a quantidade de melodias geradas definidas com cada sensação para cada opção de parâmetro.

A motivação deste trabalho reside em auxiliar compositores em processos criativos possibilidades geradas que podem ser aproveitadas em composições completas, e possibilitar um formato de busca de dados concretos que relacionam a experiência do ouvinte com a teoria utilizada na composição musical. Na Figura 3, estão destacadas as etapas do desenvolvimento do projeto, sendo elas a estruturação geral, implementação da geração de melodias, implementação de coleta de avaliações, coleta de dados destas avaliações e implementação das análises de dados coletados.

Figura 3 – Etapas de desenvolvimento do projeto



Fonte: Autoria própria (2024).

3.2.1 Estrutura do projeto

O primeiro passo da estruturação será a criação do banco de dados seguindo o DER desenvolvido. Após isso, será efetuada a implementação de uma Application programming interface (API) para interação com este banco de dados, com operações de inserção e leitura para cada tabela de banco. Para testar a API desenvolvida, são utilizadas requisições HTTP com a ferramenta Postman, visto que o sistema ainda não dispõe de uma interface gráfica para isto. As tabelas do banco de dados mencionadas, de melodia e avaliação, serão mapeadas por entidades na API, tipos de arquivos que mapeiam atributos e criam tabelas de banco de dados correspondendo aos atributos definidos. A melodia contém um atributo identificador, seu parâmetro de batimentos por minuto, sua escala musical, seu instrumento, suas oitavas utiliza-

das, seus intervalos, sua melodia, e a data em que foi gerada. Ademais, será implementada também a entidade avaliação, contendo um atributo identificador, uma nota, uma sensação e o identificador da melodia correspondente.

3.2.2 Implementação de geração de melodias

A geração de melodias foi implementada a partir de uma classe geradora de melodias, dispondo de um método principal gerar melodia, que é responsável por criar uma melodia com base em parâmetros como instrumento, escala, Batimentos por minuto (BPM) e variação de oitavas, além de parâmetros fixos para a oitava inicial, repetições máximas de cada nota, e duração total permitida. Na Listagem 1, está uma descrição do código utilizado.

Listagem 1 – Pseudocódigo - Gerar melodia

```
1
2  Função gerarMelodia(instrumento, escala, bpm, oitavaInicial, variacaoOitavas):
3      notasEscala = obter notas da escala
4      repeticoesNotas = criar mapa para contar repetições de cada nota
5      repeticoesMaximas = 4
6      duracaoTotalSegundos = 15
7      segundosPorBatida = 60 / bpm
8      melodia = "I[" + instrumento + "] "
9      tempoTotalAcumulado = 0
10
11     Enquanto tempoTotalAcumulado < duracaoTotalSegundos:
12         notasDisponiveis = notas não excedendo repeticoesMaximas
13         Se notasDisponiveis estiver vazia
14           interromper
15
16         nota = escolher nota aleatória de notasDisponiveis
17         oitavaAleatoria = oitavaInicial + aleatorio(0, variacaoOitavas)
18         nota = nota + oitavaAleatoria
19         duracaoSorteada = escolha aleatória de {1, 2, 4, 8, 16}
20         duracaoNota = converter duracaoSorteada para representacao musical
21         tempoNota = segundosPorBatida * (4 / duracaoSorteada)
22
23         Se tempoTotalAcumulado + tempoNota > duracaoTotalSegundos
24           interromper
25
26         melodia += nota + duracaoNota + " "
27         atualizar repeticoesNotas para nota sem oitava
28         tempoTotalAcumulado = tempoTotalAcumulado + tempoNota
29
30     Retornar melodia
31
32
33
```

Fonte: Autoria própria (2025).

A função gerarMelodia() retorna uma variável de texto que representa a melodia no formato interpretável pela biblioteca JFugue. Para a execução deste método, são recebidos cinco parâmetros: instrumento, que define o instrumento musical utilizado; escala, um objeto que armazena as notas disponíveis; bpm, que define os batimentos por minuto da melodia, por consequência sua velocidade, oitava inicial, que indica a oitava base para geração das notas e variação de oitavas, que especifica a variação permitida para as oitavas. O processo começa inicializando variáveis importantes. A lista de notas da escala é obtida via um enumerador, classe externa contendo essa informação. Um mapa é criado para armazenar quantas vezes cada nota foi usada, sendo inicializado com valor 0 para cada nota. Duas constantes são definidas: sendo elas o valor de repetições máximas como 4, que limita o número de vezes que uma mesma nota pode ser repetida, e duração total em segundos como 15, que determina a duração total da melodia. Estes valores estão definidos fixamente no código, como parte das heurísticas de geração. O valor de segundos por batida é calculado dividindo 60.0 pelo parâmetro batimentos por minuto. Um objeto de texto contendo a melodia é inicializado com a base da melodia "I[instrumento]", enquanto a variável de tempo total acumulado é definida como 0.0. A geração da melodia ocorre em um laço que se repete enquanto o tempo total for menor que duração total de segundos e ainda hajam notas disponíveis no mapa iniciado. Uma nota é escolhida aleatoriamente da lista de notas disponíveis, e uma oitava é gerada somando a oitava inicial com um valor aleatório entre 0 e o parâmetro variação de oitavas. Essa oitava é concatenada à nota. Em seguida, uma duração para a nota é sorteada a partir dos valores 1, 2, 4, 8, 16, e a representação textual dessa duração é obtida com uma função auxiliar. O tempo da nota é calculado multiplicando os segundos por batida da melodia pela divisão da duração sorteada por 4.0, sendo este valor fixo referente ao maior tempo possível dentre os sorteados, o "semibreve". Caso a nota sorteada não caiba no tempo restante, o laço é encerrado, em contrapartida, se ela couber, será adicionada ao objeto de texto que contém a melodia e o tempo da nota será somado ao tempo total. Ao final, o método retorna o objeto que contém a melodia.

O teste é feito pelo postman inicialmente, recebendo os parâmetros por um ponto de acesso Hyper Text Transfer Protocol (HTTP) em uma classe controller, mapeado pela Uniform Resource Locator (URL) "/geracao", além deste, também é criado um ponto de acesso para buscar uma melodia pelo seu id, que responde com um Data Transfer Object (DTO) contendo os bytes referentes a melodia buscada em formato Musical Instrument Digital Interface (MIDI), o seu id, e seu bpm. Esta classe está descrita na Listagem 4.

Listagem 2 – Pseudocódigo - Controller Geracao melodia

```
1
2  @Controller
3  @Mapeamento("/geracao")
4  GeracaoController
5      Definir dependência para serviço de melodias
6
7      Método POST
8      Retorna resposta http contendo long
9      generateMelodia(Parametros)
10         melodia = melodiaService.gerarMelodia(parametros)
11         Retornar resposta http OK com o ID da melodia
12
13     Método GET
14     Retorna resposta http contendo objeto getMidiMelodia(id)
15         melodia = melodiaService busca por id(id)
16
17         Criar arquivo temporário midiFile com nome "melodiaTemp{id}.midi"
18
19         Salvar melodia no arquivo MIDI
20         midiBytes = Ler bytes do arquivo
21
22         Criar objeto resposta com:
23             ID da melodia
24             midiBytes
25             BPM da melodia
26
27         Retornar resposta http OK com objeto response
28
29
30
```

Fonte: Autoria própria (2025).

Posteriormente, é implementado um componente em uma aplicação web desenvolvida com o framework React, disposto de caixas de combinação contendo todas as opções disponíveis de geração para escala e instrumentos, e dois campos de texto limitados a valores numéricos, onde são digitados a quantidade de oitavas permitidas e os batimentos por minuto, os campos de texto dispõe de uma verificação para que o usuário não possa desobedecer a heurística de geração que limita o parâmetro de oitavas a valores entre 1 e 8 e o parâmetro de batimentos a valores entre 40 e 300. Além disso, a tela contém um botão de gerar melodia, que coleta as informações dos campos, constrói e envia a requisição HTTP ao controller mencionado anteriormente, recebendo e armazenando o id da melodia gerada.

3.2.3 Implementação de avaliações

Neste momento, é implementada a avaliação das melodias geradas, onde a classe avaliação implementada é utilizada para associar uma avaliação a uma melodia gerada, contendo um atributo para sua identificação, um para nota, um para a sensação escolhida e um para a identificação da melodia que recebe esta avaliação, na Listagem 3, é demonstrada a avaliação.

Listagem 3 – Pseudocódigo - Classe Avaliação

```
1
2  @Tabela(nome = "avaliacao")
3  @Entidade
4  Classe Avaliacao:
5
6
7      Atributo id: inteiro longo
8      Atributo nota: inteiro
9      Atributo sensacao: enumerador  Sensação
10     Atributo melodia: objeto do tipo Melodia
11
12
```

Fonte: Autoria própria (2025).

A classe também contém uma anotação com o nome da tabela, que define o nome com o qual a tabela será criada e manipulada no banco de dados, uma anotação entidade, que classifica a mesma como tal, permitindo a sua associação ao banco de dados. O atributo id é de tipo long, o de nota de tipo inteiro, já a sensação é associada a um enumerador, que mapeia a mesma às possibilidades definidas por um arquivo enum, por fim, o atributo de melodia recebe uma classe melodia como seu tipo, como esta classe é mapeada como entidade no código, este atributo é associado ao id da melodia em questão quando está no banco de dados, mas permite manipulação do objeto completo dentro do sistema java. Esta avaliação é recebida por meio de um ponto de acesso HTTP que recebe as informações da avaliação por meio de um DTO numa classe controller, mapeada pela URL "/avaliacao", demonstrado na Listagem 4.

Listagem 4 – Pseudocódigo - Controller Geracao melodia

```
1
2  @Controller
3  @Mapeamento("/geracao")
4  Classe AvaliacaoController
5
6    Definir dependência para serviço de avaliações
7
8      Método POST
9      Resposta http receiveAvaliacao(avaliacao)
10         Salvar os dados recebidos na avaliação
11         Retornar resposta http OK
12
13
```

Fonte: Autoria própria (2025).

Para o teste, inicialmente é gerada uma melodia pela parte do sistema já desenvolvida e guardado o id da mesma, com este id, é possível utilizar o postman para enviar uma requisição ao ponto de acesso mapeado, contendo a avaliação. Posteriormente, é implementada uma página de avaliação no sistema, que busca as informações da melodia gerada anteriormente e dispõe contendo um player para a melodia gerada, uma opção de download da melodia em formato midi e os parâmetros para a avaliação. Estes parâmetros são dispostos por uma sequência de estrelas utilizada para definir a nota da melodia de 0 a 10, e uma caixa de combinação contendo as opções de sensações disponíveis para descrever a melodia. Por fim, um botão de avaliar melodia permite ao usuário mandar os parâmetros para o ponto de acesso, salvando sua avaliação. Ao enviar a avaliação, a tela fecha a seção de avaliação e seus parâmetros, mantendo apenas o player da melodia e a opção de download. Após a implementação desta página, a tela de geração de melodias passa a direcionar o usuário para a mesma, com as informações da melodia recém gerada.

3.2.4 Implementação de análises

Esta é a etapa final, onde são desenvolvidas as análises das avaliações armazenadas no sistema, fazendo pesquisas no banco de dados para coletar estatísticas relacionadas às mesmas. Estas estatísticas incluem quais opções de cada parâmetro de geração geram as melodias melhor avaliadas, e quais opções geram melodias de cada sensação, trabalhando cada parâmetro separadamente. Estas pesquisas são implementadas em uma interface repository, que recebe uma entidade na sua criação e permite manipular a tabela referente a esta entidade via comandos Structured Query Language (SQL), neste caso, os comandos são implementados no repository que recebe a entidade melodia, uma descrição desta interface e das pesquisas está disposta na Listagem 5

Listagem 5 – Pseudocódigo - Repositório de Melodias e análises

```
classe RepositorioMelodias(Melodia)

  buscarQuantidadePorParametroESensacao(parametro, sensacao)
    Definir consulta que:
      Quando o parametro for "BPM",
      agrupar em intervalos de 10 unidades
      Quando o parametro for "Escala",
      usar diretamente o valor da escala
      Quando o parametro for "Instrumento",
      usar diretamente o valor do instrumento
      Quando o parametro for "Oitavas",
      usar diretamente o valor das oitavas
      Caso seja outro parametro, retornar "Desconhecido"
    Filtrar os resultados para incluir
    apenas avaliações com a sensação informada
    Contar quantas vezes cada valor aparece
    Ordenar em ordem decrescente de quantidade
    Retornar lista de grupos com seus contadores

  buscarQuantidadePorParametroENota(parametro, notaMinima, notaMaxima)
    Definir consulta que:
       Quando o parametro for "BPM",
       agrupar em intervalos de 10 unidades
       Quando o parametro for "Escala",
       usar diretamente o valor da escala
       Quando o parametro for "Instrumento",
       usar diretamente o valor do instrumento
       Quando o parametro for "Oitavas",
       usar diretamente o valor das oitavas
       Quando o parametro for "Sensacao",
       usar diretamente a sensação da avaliação
       Caso seja outro parametro, retornar "Desconhecido"
    Filtrar os resultados para incluir
    apenas avaliações com nota dentro do intervalo informado
    Contar quantas vezes cada valor aparece
    Ordenar em ordem decrescente de quantidade
    Retornar lista de grupos com seus contadores
```

Fonte: Autoria própria (2025).

A pesquisa por parâmetro e sensação recebe qual parâmetro e qual sensação deverá usar para fazer a busca, buscando e retornando, para cada opção dentro daquele parâmetro, a quantidade de avaliações feitas com a sensação escolhida para descrever a melodia, retornando de forma agrupada. Para o parâmetro de batimentos por minuto, pesquisará por cada faixa de 10 dentro das opções, para o parâmetro escala, pesquisará por cada uma das escalas disponíves,

para o parâmetro instrumento, pesquisará por cada um dos instrumentos disponíveis, e, para o parâmetro de oitavas, pesquisará por cada opção de escala permitida.

No caso da pesquisa por parâmetro e nota, são recebidos qual parâmetro deverá ser usado para a pesquisa, qual a nota mínima, e qual a nota máxima, retornando, para cada opção daquele parâmetro, a quantidade de avaliações feitas com notas iguais ou entre a nota mínima e máxima fornecida. Para o parâmetro de batimentos por minuto, pesquisará por cada faixa de 10 dentro das opções, para o parâmetro escala, pesquisará por cada uma das escalas disponíves, para o parâmetro instrumento, pesquisará por cada um dos instrumentos disponíveis, para o parâmetro de oitavas, pesquisará por cada opção de escala permitida, nesta pesquisa é também usado o parâmetro de sensação, que pesquisará por cada opção de sensação disponível para avaliação.

Essas pesquisas são acessíveis por pontos de acesso HTTP para cada uma das pesquisas, definidos em uma classe controller designada apenas para análises e mapeado pela URL "/analises", demonstrado na Listagem 6.

Listagem 6 – Pseudocódigo - Controller de Análises

```
1
2  @Controller
3  @Mapeamento("/analises")
4  Classe AnaliseController
5
6    Definir dependência para serviço de melodias
7
8      Método GET
9      Lista de objetos de resultado da análise  obterAnalisePorSensacaoEParametro(dto)
10         Chamar o serviço de melodias para realizar a análise
11         Retornar lista com resultados da análise
12
13     Método GET
14     Lista de objetos de resultado da análise obterAnalisePorNotaEParametro(dto)
15         Chamar o serviço de melodias para realizar a análise
16         Retornar lista com resultados da análise
17
18
19
```

Fonte: Autoria própria (2025).

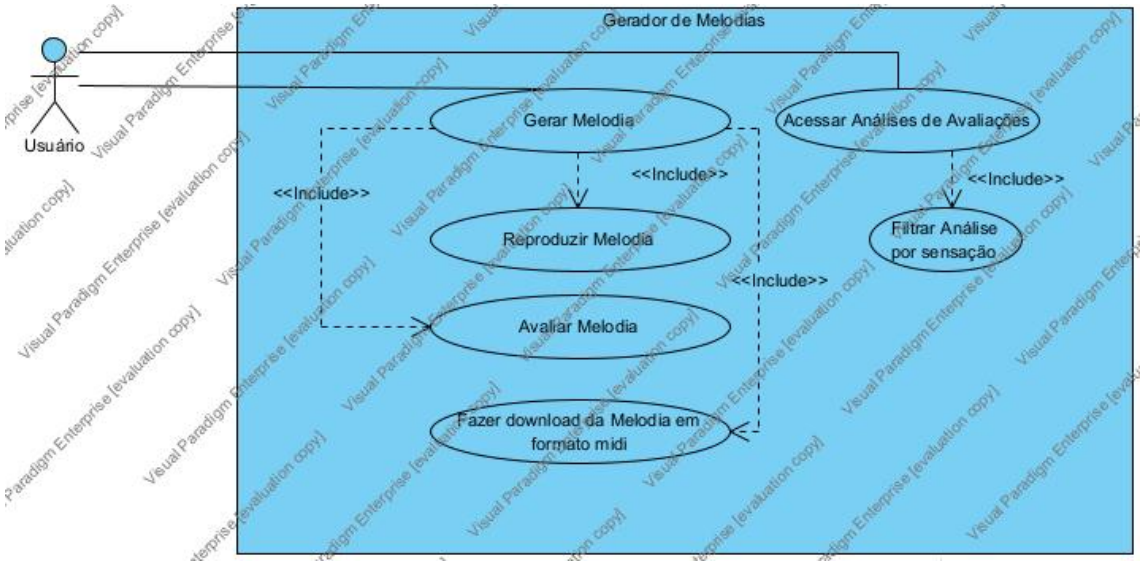
Inicialmente, o teste destes pontos de acesso são realizados via requisições HTTP utilizando o Postman, fornecendo o parâmetro e as notas mínimas e máximas ou sensação a serem pesquisados, observando se os agrupemtos de resultado estão corretos. Após isto, é implementada uma página de análises no sistema, acessível ao usuário a qualquer momento em sua navegação, esta página é carregada inicialmente utilizando a pesquisa por notas e parâmetros, demonstrando gráficos visuais para cada opção de parâmetros que apresentam notas altas (7-10). Esta página dispõe de uma caixa de combinação com cada uma das sensações

de avaliação disponível, permitindo ao usuário filtrar a pesquisa para o formato de sensações e parâmetro, e demonstrando os gráficos referentes a cada opção de parâmetro para a sensação escolhida.

3.2.5 Diagrama de casos de uso

Na Figura 4, é apresentado o diagrama de casos de uso do sistema, ilustrando as interações possíveis entre o ator, o usuário, e as funcionalidades do sistema. O diagrama abrange a geração de uma melodia, sua reprodução, avaliação, download em formato .mid, e o acesso às análises e seus filtros. Cada um desses casos de uso descreve uma sequência de ações realizadas pelo usuário, permitindo que ele interaja com o sistema de forma a criar, avaliar e obter informações sobre as melodias geradas.

Figura 4 – Diagrama de Casos de Uso



Fonte: Autoria própria (2025).

O caso de uso Gerar Melodia descreve o processo pelo qual um usuário pode criar uma melodia a partir de parâmetros personalizados. Para isso, ele deve acessar a página de geração de melodias e definir aspectos como escala, instrumento, BPM e oitavas desejadas. Após o envio dessas configurações, o sistema processa os dados, gera a melodia correspondente e a armazena no banco de dados. Em seguida, o usuário é direcionado automaticamente para a página de avaliação, onde pode ouvir a melodia gerada, avaliá-la e realizar o download. Se o usuário optar por não gerar uma nova melodia, ele pode acessar a seção de análises por meio de um link disponível na barra superior. Além disso, há a possibilidade de falha caso o sistema esteja indisponível, momento em que uma mensagem de erro será exibida ao usuário. A Tabela 3 apresenta esse caso de uso em detalhes.

Tabela 3 – Caso de uso - Gerar melodia

Nome do Caso de Uso	Gerar Melodia
Ator	Usuário
Descrição	O caso de uso descreve o processo de um usuário fornecendo parâmetros e gerando uma melodia
Pré-condições	O usuário deverá fornecer todos os parâmetros solicitados.
Pós-condições	O usuário é direcionado a página de avaliação, onde pode ouvir, avaliar e baixar a melodia gerada.
Fluxo Principal	1.[in] O usuário acessa a página de geração de melodia. 2.[in] O usuário seleciona os parâmetros de escala, instrumento, bpm e oitavas desejados. 3.[out] O sistema processa os parâmetros e gera uma melodia, inserindo-os no banco de dados. 4.[out] O sistema direciona o usuário a página de avaliação da melodia gerada.
Fluxos Alternativos	1. Se o usuário não desejar gerar uma melodia, ele pode clicar no link "Análises"na barra superior, e ser direcionado para a página de análises.
Exceções	1. O sistema está fora do ar e a geração falha, demonstrando um erro ao usuário.

Fonte: Autoria própria (2025).

O caso de uso Reproduzir Melodia descreve o processo pelo qual um usuário pode ouvir a melodia gerada. Para isso, ele deve ter previamente criado uma melodia e sido direcionado à página de avaliação. Na interface dessa página, o usuário pode iniciar a reprodução clicando no botão do player disponível. Ao fazer isso, a melodia será tocada diretamente pelo navegador, acompanhada por uma representação visual do progresso da reprodução. Se o usuário optar por não avaliar a melodia naquele momento, ele pode acessar a seção de análises através de um link na barra superior. Além disso, caso prefira gerar uma nova melodia em vez de interagir com a atual, pode retornar à tela de geração. Entretanto, se houver um problema na reprodução do arquivo MIDI, o sistema exibirá uma mensagem de erro para o usuário. A Tabela 4 apresenta esse caso de uso em detalhes.

Tabela 4 – Caso de uso - Reproduzir melodia

Nome do Caso de Uso	Reproduzir Melodia
Ator	Usuário
Descrição	O caso de uso descreve o processo de um usuário reproduzindo a melodia gerada
Pré-condições	O usuário deverá ter gerado uma melodia e sido direcionado a página de avaliação.
Pós-condições	O usuário ouvirá a melodia tocada pelo player.
Fluxo Principal	1.[in] O usuário clica no botão de reprodução do player em tela. 2.[out] O usuário ouvirá a melodia pelo seu navegador e verá uma representação visual do seu progresso de reprodução.
Fluxos Alternativos	1. Se usuário não deseja avaliar a melodia, ele pode clicar no link "Análises"na barra superior, e ser direcionado para a página de análises. 2. Se usuário não deseja interagir com a a melodia, ele pode clicar no link "Geração"para ir a tela de geração.

Exceções	1. O sistema encontra um problema na reprodução do arquivo midi no player, demonstrando um ero ao usuário.

Fonte: Autoria própria (2025).

O caso de uso Avaliar Melodia descreve o processo pelo qual um usuário pode fornecer uma avaliação para a melodia gerada. Para isso, ele deve ter previamente criado uma melodia e sido direcionado à página de avaliação. Nessa página, o usuário pode atribuir uma nota e indicar a sensação que a melodia lhe transmite. Após o envio desses dados, o sistema os armazena no banco de dados e fecha o formulário de avaliação, mantendo apenas as opções de reprodução e download da melodia. Se o usuário optar por não avaliar a melodia, ele pode acessar a seção de análises por meio de um link na barra superior ou retornar à tela de geração de melodias. Caso ocorra uma falha na conexão com o back-end, o sistema informará o erro ao usuário. A Tabela 5 apresenta esse caso de uso em detalhes.

Tabela 5 – Caso de uso - Avaliar melodia

Nome do Caso de Uso	Avaliar Melodia
Ator	Usuário
Descrição	O caso de uso descreve o processo de um usuário avaliando a melodia gerada
Pré-condições	O usuário deverá ter gerado uma melodia e sido direcionado a página de avaliação e fornecer os parâmetro de nota e sensação.
Pós-condições	O sistema salvará a avaliação fornecida pelo usuário em banco de dados.
Fluxo Principal	1.[in] O usuário fornece os parâmetros de nota e sensação para a melodia em avaliação. 2.[out] o sistema salva a avaliação no banco de dados. 3.[out] o sistema fecha o formulário de avaliação da tela, mantendo apenas as opções de ouvir e baixar.
Fluxos Alternativos	1. Se usuário não deseja avaliar a melodia, ele pode clicar no link "Análises"na barra superior, e ser direcionado para a página de análises. 2.Se usuário não deseja avaliar a melodia, ele pode clicar no link "Geração"para ir a tela de geração.
Exceções	1. O sistema encontra um problema na conexão com o back-end, informando o erro ao avaliar a melodia para o usuário.

Fonte: Autoria própria (2025).

O caso de uso Baixar Melodia descreve o processo pelo qual um usuário pode efetuar o download da melodia gerada. Para realizar essa ação, ele deve ter criado previamente uma melodia e sido direcionado à página de avaliação. Nessa página, ao clicar no botão de download, o sistema disponibiliza o arquivo no formato .mid para ser baixado diretamente pelo navegador do usuário. Caso o usuário opte por não baixar a melodia, ele pode acessar a página de análises por meio de um link na barra superior ou retornar à tela de geração de melodias. Se houver um bloqueio de downloads no navegador, a operação será impedida. A Tabela 6 apresenta esse caso de uso em detalhes.

Tabela 6 – Caso de uso - Baixar melodia como .mid

Nome do Caso de Uso	Baixar Melodia

Ator	Usuário
Descrição	O caso de uso descreve o processo de um usuário fazendo download da melodia gerada
Pré-condições	O usuário deverá ter gerado uma melodia e sido direcionado a página de avaliação.
Pós-condições	O usuário terá efetuado download do arquivo .mid da melodia.
Fluxo Principal	1.[in] O usuário clica no botão de download em tela. 2.[out] O usuário fará download do arquivo em seu navegador.
Fluxos Alternativos	1. Se usuário não deseja interagir com a melodia, ele pode clicar no link "Análises"na barra superior, e ser direcionado para a página de análises. 2. Se usuário não deseja interagir com a a melodia, ele pode clicar no link "Geração"para ir a tela de geração.
Exceções	1. O usuário possui um bloqueio de downloads em seu navegador, impedindo-o de baixar a melodia.

Fonte: Autoria própria (2025).

O caso de uso Análise de dados de avaliações descreve o processo pelo qual um usuário acessa as análises das avaliações armazenadas no sistema. Este caso de uso pode ser acessado a qualquer momento durante a execução do sistema, pois não possui pré-condições. Após clicar no botão de análise disponível na barra superior, o usuário é direcionado para a tela de análise, onde pode visualizar gráficos que mostram as avaliações por nota. Se houver um problema de comunicação com o back-end, o sistema pode apresentar gráficos vazios devido à falta de informações. A Tabela 7 apresenta esse caso de uso em detalhes.

Tabela 7 – Caso de uso - Acesso a tela de análise

Nome do Caso de Uso	Análise de dados de avaliações
Ator	Usuário
Descrição	O caso de uso descreve o processo do usuário acessando as análises de dados de avaliação armazenados no sistema
Pré-condições	Este caso de uso não possui pré condições, as análises são disponíveis a qualquer momento da execução do sistema.
Pós-condições	O usuário verá gráficos acerca das avaliações armazenadas, filtrados por avaliações.
Fluxo Principal	1.[in] O usuário clica no botão de análise na barra superior. 2.[out] O usuário é direcionado a tela de análise, que permite ver gráficos de avaliações por nota.
Fluxos Alternativos	-
Exceções	1. O sistema encontra um problema de comunicação com o back-end, demonstrando gráficos vazio devido a falta de informações.

Fonte: Autoria própria (2025).

O caso de uso filtro de análises de avaliações descreve o processo no qual o usuário acessa as análises filtradas de dados de avaliação. Para acessar esse caso de uso, o usuário precisa estar na página de análises. Após selecionar o parâmetro de sensação desejado em uma caixa de combinação disponível na tela, o sistema exibe gráficos na tela de análise, mos-

trando os parâmetros mais utilizados na geração de melodias, classificados de acordo com a sensação escolhida. Caso haja problemas de comunicação com o back-end, os gráficos podem ser exibidos de forma vazia devido à falta de informações. A Tabela 8 apresenta esse caso de uso com mais detalhes.

Tabela 8 – Caso de uso - Acesso a tela de análise

Nome do Caso de Uso	Análise de dados de avaliações
Ator	Usuário
Descrição	O caso de uso descreve o processo do usuário acessando as análises de dados filtradas
Pré-condições	O usuário deverá estar na página de análises.
Pós-condições	O usuário verá gráficos acerca das avaliações armazenadas, filtrados pela sensação escolhida.
Fluxo Principal	1.[in] O seleciona o parâmetro de sensação, disponível numa caixa de combinação na tela. 2.[out] São demonstrados na tela de análise os gráficos demonstrando os parâmetros mais utilizadas na geração de melodias classificadas pela sensação selecionada.
Fluxos Alternativos	-
Exceções	1. O sistema encontra um problema de comunicação com o back-end, demonstrando gráficos vazio devido a falta de informações.

Fonte: Autoria própria (2025).

4 RESULTADOS

Neste capítulo, serão abordados os resultados gerados pelo trabalho após sua implementação. Inicialmente será exposto o escopo do sistema, que comenta as principais funcionalidades, a modelagem de banco de dados, então, uma demonstração do sistema, suas páginas e sua implementação e por fim, disponibiliza o código fonte do projeto.

4.1 Escopo do sistema

O sistema é acessado como um site WEB, que direciona o usuário a página de geração, onde o mesmo deverá escolher os parâmetros sobre a escala a ser utilizada na geração, o instrumento em que a melodia será tocada, quantas oitavas a geração poderá utilizar, e as batidas por minuto; em seguida, o usuário pode clicar no botão gerar, para que o sistema crie a melodia acerca dos parâmetros escolhidos.

Após a geração, o usuário será direcionado para uma página de avaliação com um player para ouvir e avaliar a melodia gerada, permitindo efetuar o download da melodia em formato .midi, e escolher uma sensação entre alegria, tristeza, inspiração, nostalgia, relaxamento e energização, decidindo qual delas mais se assemelha ao que sentiu ao ouvir a melodia. A melodia gerada, juntamente com uma descrição dos intervalos das notas dentro da escala, a data de geração e os parâmetros do usuário serão salvos em banco de dados em uma tabela do banco de dados chamada melodia, enquanto a avaliação é salva separadamente em uma tabela própria.

Por fim, haverá uma página de análise, que consumirá as informações salvas em banco de dados sobre as avaliações das melodias geradas e mostrará correlações entre parâmetros utilizados e fatores sobre as melodias geradas, que, segundo usuários, causam uma determinada sensação, e, além disso, trouxe avaliações mais positivas. Essas análises serão feitas quantitativamente, no sentido que, serão estatísticas acerca das informações consumidas do banco de dados.Estas informações serão demonstradas tanto de forma direta, com simples demonstração de observações dos dados, quanto demonstração de dados em gráficos, exibindo quais dos parâmetros citados anteriormente revelam avaliações de nota mais alta, e são comumente descritos com cada sensação. será possível ter uma visão geral de dados ou filtrar as análises por sensação.

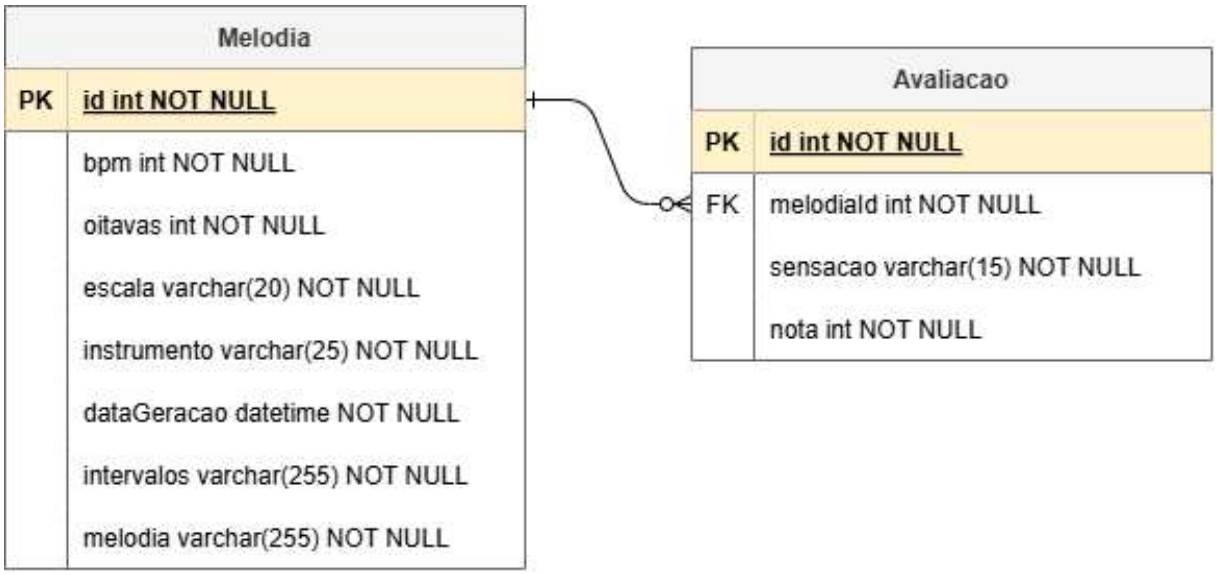
4.2 Modelagem do sistema

Neste capítulo, estão demonstradas as características da modelagem do sistema.

4.2.1 Diagrama de Entidade Relacionamento DER

Na Figura 5, é apresentado o diagrama de entidade e relacionamento referente ao banco de dados do sistema.

Figura 5 – Diagrama de entidade Relacionamento



Fonte: Autoria Própria(2025).

Conforme representado no diagrama da Figura 5 a tabela Melodia contém os parâmetros de geração e a geração em si de melodias, incluindo um relacionamento de 1:1 com a tabela de avaliação, responsável por armazenar a avaliação de cada melodia pelo usuário. A seguir, na tabela Tabela 10 e Tabela 9 são detalhadas a estrutura e atributos de cada tabela.

4.2.2 Tabelas de Banco de Dados

Tabela 9 – Estrutura da Tabela de Melodias

Campo	Tipo	Nulidade	Primária	Estrangeira	Valor Padrão
id	bigint	NÃO	PRI	-	auto_-increment
bpm	int	NÃO	-	-	NULL
escala	varchar(20)	NÃO	-	-	NULL
instrumento	varchar(25)	NÃO	-	-	NULL
oitavas	int	NÃO	-	-	NULL
intervalos	varchar(255)	NÃO	-	-	NULL
melodia	varchar(255)	NÃO	-	-	NULL
data_-geracao	datetime(6)	NÃO	-	-	NULL

Fonte: Autoria própria (2025).

- id - Atributo de tipo inteiro, é a chave primária da tabela, é gerado automaticamente, garantindo que cada linha inserida possua um identificador único.
- bpm - Atributo de tipo inteiro que receberá o valor de batidas por minuto da melodia gerada, aceita valores de 40 a 300.
- escala - Atributo de tipo texto, recebe a escala musical escolhida para a geração, limitada a 20 caracteres, visto que as opções disponibilizadas no sistema, que incluem C(dó), D(ré), E(mi), F(fá), G(sol), A(lá), B(si), suas variações menores, e seus acidentes sustenidos, não permitem opções que ocupem mais do que este tamanho.
- instrumento - Atributo de tipo texto, recebe o instrumento escolhido para a melodia, limitado a 25 caracteres pois as opções disponibilizadas no sistema, que incluem Piano, Violão, Guitarra, Baixo e Violino, não ultrapassam este tamanho no formato em que serão salvos.
- oitavas - Atributo do tipo inteiro, recebe a quantidade de oitavas escolhida pelo usuário para ser permitida na geração.
- intervalos - Atributo de tipo texto, recebe a contagem de uso de cada intervalo musical nas melodias geradas.
- melodia - Atributo do tipo texto, recebe o texto referente a melodia gerada no formato aceito pela biblioteca JFugue.
- data geracao - Atributo do tipo data, recebe a data em que a melodia foi gerada.

A seguir, na tabela Tabela 10 é detalhada a tabela de avaliações.

Tabela 10 – Estrutura da Tabela de Avaliações

Campo	Tipo	Nulidade	Chave Primária	Chave Estrangeira	Valor Padrão
id	bigint	NÃO	PRI	-	auto_increment
nota	int	SIM	-	-	NULL
sensacao	varchar(15)	NÃO	-	-	NULL
id_melodia	bigint	NÃO	UNI	SIM	NULL

Fonte: Autoria própria (2025).

- id - Atributo de tipo inteiro, é a chave primária da tabela, é gerado automaticamente, garantindo que cada linha inserida possua um identificador único.
- nota - Atributo de tipo inteiro que receberá a nota de 0 a 10 fornecida pelo usuário ao avaliar a melodia.

- sensacao - Atributo de tipo texto, recebe a sensação escolhida pelo usuário para descrever a melodia gerada, limitada a 15 caracteres, visto que as opções disponibilizadas no sistema, que incluem Alegre, Triste, Inspirado, Nostálgico, Relaxado e Energizado, não permitem mais do que este tamanho.
- id melodia - Atributo de tipo inteiro, este é o atributo de chave estrangeira que relaciona a tabela de avaliações com a tabela de melodias, armazenando a chave primária da linha da tabela de melodias, na tabela de avaliações, permitindo futura referência entre a linha de uma a outra.

4.3 Demonstração do sistema

Nesta seção, é apresentado o funcionamento da camada visual do sistema, considerando a utilização pelo próprio usuário e sua experiência durante o ciclo de vida padrão do sistema, demonstrando o desenvolvimento front-end do mesmo. A Figura 6 demonstra a página de geração de melodias do sistema.

Figura 6 – Tela de Geração



Fonte: Autoria Própria (2025).

Nesta página, o usuário fornece os parâmetros para geração da melodia, e clica no botão gerar, sendo eles os seguintes:

- Instrumento - Uma caixa de seleção contendo as opções de instrumento oferecidas pelo sistema, sendo eles Piano, Violão, Guitarra, Baixo e Violino.
- Escala - Uma caixa de seleção contendo as opções de escala musical para geração, contendo as escalas maiores C(dó), D(ré), E(mi), F(fá), G(sol), A(lá), B(si), suas variações menores, e seus acidentes sustenidos.

- Oitavas - Um campo de texto que permite digitação de valores de 1 a 8, referente a quantidade de oitavas permitidas para uso nessa geração.
- BPM - Um campo de texto que permite digitação de valores de 40 a 300, referente as batidas por minuto da melodia gerada.

A Figura 7 demonstra a página de avaliação de melodias do sistema. Nesta página, o usuário tem a opção de ouvir a melodia gerada pelo player, ou baixá-la como arquivo .midi. Além disso, o usuário tem a opção de avaliar a melodia após ouvi-la, fornecendo uma nota de 0 a 10, clicando nas estrelas em tela, e uma sensação que descreve a melodia, permitindo escolher entre Alegre, Triste, Inspirado, Nostálgico, Relaxado e Energizado.

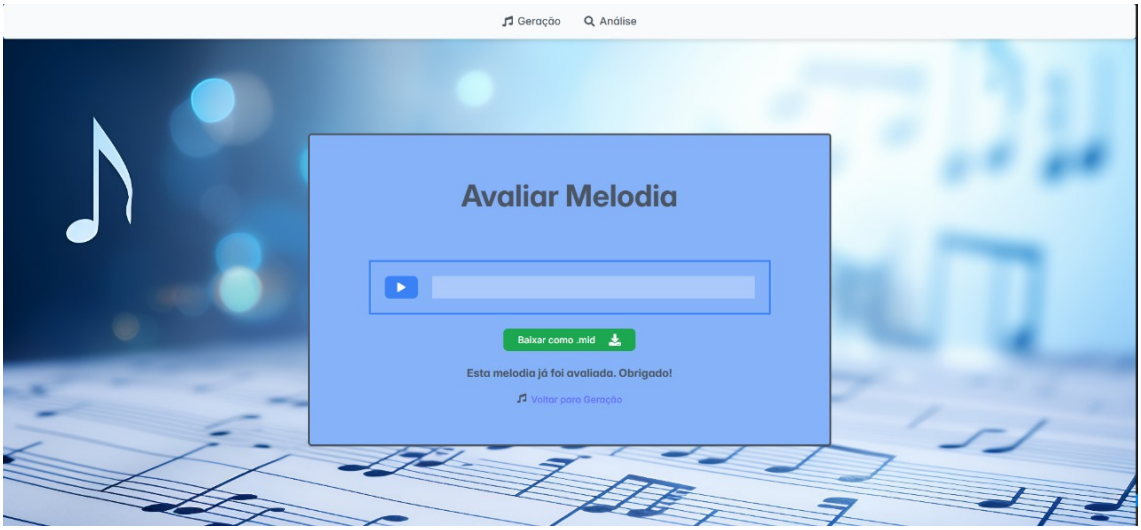
Figura 7 – Tela de Avaliação



Fonte: Autoria Própria (2025).

A Figura 7 demonstra a página de avaliação de melodias após a avaliação da melodia gerada. Esta tela permite que o usuário ouça a melodia pelo player na própria página, e faça download do midi, todavia, nesta versão da página, não é mostrado formulário de avaliação, trocando o mesmo por um botão que redireciona o usuário a tela de geração de melodias.

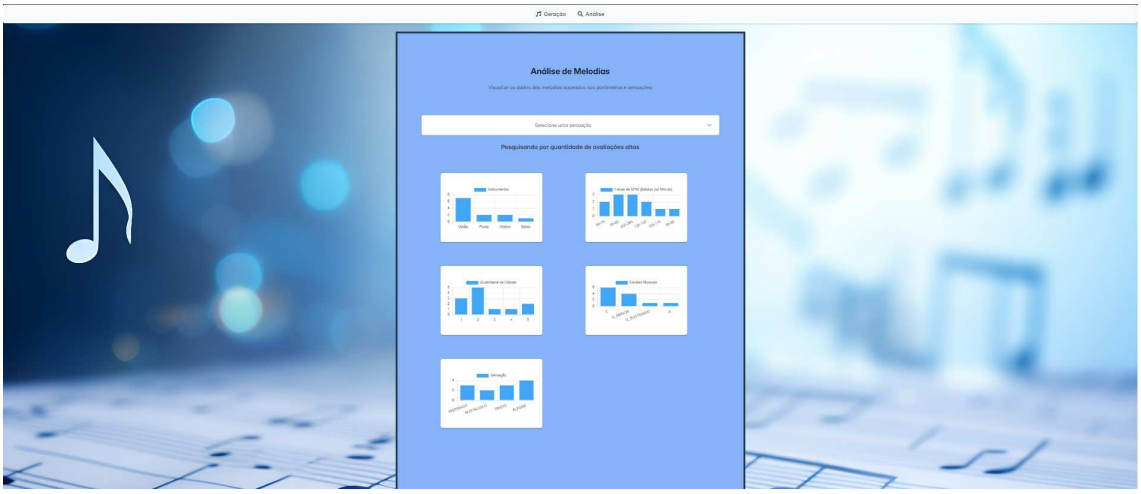
Figura 8 – Tela de Avaliação após avaliar



Fonte: Autoria Própria (2025).

A Figura 9 demonstra a página de análise de avaliações, disponível ao usuário a qualquer momento no sistema pela barra superior, no botão Análise.

Figura 9 – Tela de Análise



Fonte: Autoria Própria (2025).

Nesta página, o usuário vê gráficos referentes a quais parâmetros resultam em melodias melhores avaliadas, e também quais sensações tendem a ter melhores avaliações. Além disso, há uma caixa de seleção para aplicar filtros referentes a cada sensação nas análises. Na Figura 10, é apresentada a tela de análise após a aplicação de um filtro de sensação, demonstrando gráficos refentes a quais parâmetros de geração resultam em maior número de melodias classificadas por usuários para aquela sensação.

Figura 10 – Tela de Análise filtrada

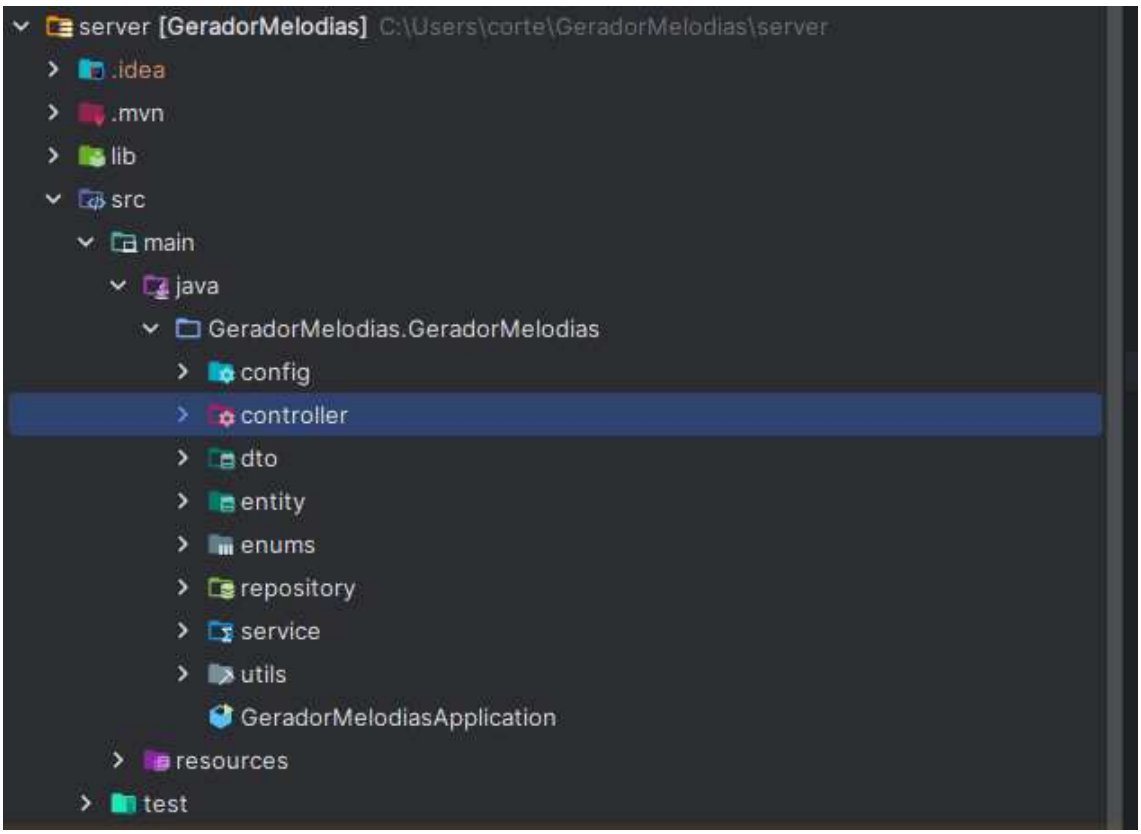


Fonte: Autoria Própria (2025).

4.4 Implementação do sistema

A seguir, está demonstrada a implementação do sistema, utilizando linguagem java para o *back-end* juntamente com o *framework* spring boot para seu funcionamento web e MySql como gerenciador de banco de dados, dentro do mesmo, já havia sido criado o schema do projeto, chamado geradorMelodias. Na Figura 11, é mostrada a organização dos pacotes no *back-end* do projeto.

Figura 11 – Organização de pacotes do backend



Fonte: Autoria Própria (2025).

A classe e o método principal para geração de melodias são denominados GeradorMelodias e gerarMelodia, respectivamente. Esse método recebe os parâmetros escolhidos pelo usuário na seção visual da geração, além do parâmetro de oitava inicial, fixado como 1 na seção de código que faz a chamada deste método. A execução começa obtendo as notas disponíveis na escala e inicializando um mapa para controlar quantas vezes cada nota foi utilizada. Define-se um limite máximo de repetições de uma mesma nota como 4 e a duração total da melodia como 15 segundos. A lógica principal ocorre dentro de um laço de repetição, que continua adicionando notas até atingir o tempo máximo de duração ou esgotar as notas disponíveis. Para cada nota, seleciona-se uma aleatoriamente entre as disponíveis e atribui-se uma oitava dentro da variação permitida. A duração da nota também é escolhida aleatoriamente a partir de valores pré-definidos. Antes de adicionar a nota à melodia, verifica-se se ela cabe no tempo restante disponível. Se sim, o tempo total da melodia é atualizado. Por fim, o método retorna a melodia gerada.

Na Listagem 7, apresenta-se a implementação dessa classe e método.

Listagem 7 – Classe GeradorMelodias - Método Principal

```java
package GeradorMelodias.GeradorMelodias.utils.GeracaoMelodias;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Random;

public class GeradorMelodias {
    private final Random random = new Random();

    public String gerarMelodia(String instrumento,
    Escala escala, int bpm,
    int oitavaInicial, int variacaoOitavas) {
        String[] notasEscala = escala.getNotas();
        HashMap<String, Integer> repeticoesNotas = new HashMap<>();
        for (String nota : notasEscala) {
            repeticoesNotas.put(nota, 0);
        }
        int repeticoesMaximas = 4;
        int duracaoTotalSegundos = 15;
        double segundosPorBatida = 60.0 / bpm; // Tempo por batida
        StringBuilder construtorMelodia =
        new StringBuilder("I[" + instrumento + "] ");
        double tempoTotalAcumulado = 0.0; // Tempo total da melodia
        while (tempoTotalAcumulado < duracaoTotalSegundos) {
            List<String> notasDisponiveis =
            obterNotasDisponiveis(notasEscala,
            repeticoesNotas, repeticoesMaximas);
            if (notasDisponiveis.isEmpty()) {
                break;
            }
            String nota =
            notasDisponiveis.get(random.nextInt(notasDisponiveis.size()));
            //Randomiza oitava da nota
            int oitavaAleatoria =
            oitavaInicial + random.nextInt(variacaoOitavas + 1);
            nota += oitavaAleatoria;
            int[] duracoes = {1, 2, 4, 8, 16};
            int duracaoSorteada = duracoes[random.nextInt(duracoes.length)];
            String duracaoNota = obterDuracaoNota(duracaoSorteada);
            //Cálculo tempo da nota
            double tempoNota = segundosPorBatida *
            (4.0 / duracaoSorteada);
            // Multiplica por 4/duração (relativo à semibreve)

            //Verifica se a nota "cabe" no tempo
            if (tempoTotalAcumulado + tempoNota > duracaoTotalSegundos) {
                break;
            }
            tempoTotalAcumulado += tempoNota; //Adiciona ao tempo total
        }
        return construtorMelodia.toString().trim();
    }


```

Fonte: Autoria própria (2025)

Na Listagem 8, são apresentados os métodos auxiliares à geração de melodia, utilizados para traduzir valores numéricos a seu respectivo símbolo de intervalo musical no formato da biblioteca JFugue, e para verificar as notas disponíveis dentro do hashMap utilizado na geração.

Listagem 8 – Classe GeradorMelodias - métodos auxiliares

```
1  package GeradorMelodias.GeradorMelodias.utils.GeracaoMelodias;
2
3
4
5      private String obterDuracaoNota(int valorNota) {
6          return switch (valorNota) {
7              case 1 -> "w"; // Semibreve
8              case 2 -> "h"; // Mínima
9              case 4 -> "q"; // Semínima
10             case 8 -> "i"; // Colcheia
11             case 16 -> "s"; // Semicolcheia
12             default ->
13             throw new IllegalArgumentException
14             ("Valor de nota inválido: " + valorNota);
15         };
16     }
17
18     private List<String> obterNotasDisponiveis
19     (String[] escala, HashMap<String,
20     Integer> repeticoesNotas, int repeticoesMaximas) {
21         List<String> notasDisponiveis = new ArrayList<>();
22         for (String nota : escala) {
23             if (repeticoesNotas.get(nota) < repeticoesMaximas) {
24                 notasDisponiveis.add(nota);
25             }
26         }
27         return notasDisponiveis;
28     }
29 }
30
31
```

Fonte: Autoria própria(2025).

A classe Melodia representa uma entidade JPA mapeada para a tabela melodia no banco de dados. Ela é anotada com @Entity e @Table(name = "melodia"), garantindo que seus atributos sejam salvos conforme definido no modelo relacional. A classe contém um identificador único id, anotado com @Id e @GeneratedValue(strategy = GenerationType.IDENTITY), permitindo que seu valor seja gerado automaticamente pelo banco de dados. Os demais atributos armazenam informações essenciais sobre a melodia gerada, incluindo melodia, que contém a sequência de notas, intervalos, que pode representar a estrutura rítmica ou intervalar da melodia, e instrumento, que registra o timbre utilizado na execução.Além disso, escala define o conjunto de notas utilizadas, oitavas indica a variação de oitavas permitida das notas, e bpm

armazena a velocidade da execução em batidas por minuto. O atributo dataGeracao, registra o momento exato em que a melodia foi criada. Para facilitar a manipulação dos dados, a classe utiliza as anotações @Getter e @Setter da biblioteca Lombok, que geram automaticamente os métodos de acesso para seus atributos. Na Listagem 9, apresenta-se a implementação dessa classe.

Listagem 9 – Classe Melodia

```
1  package GeradorMelodias.GeradorMelodias.entity;
2
3  import jakarta.persistence.*;
4  import lombok.Getter;
5  import lombok.Setter;
6  import java.time.LocalDateTime;
7
8  @Table(name = "melodia")
9  @Entity
10 @Getter
11 @Setter
12 public class Melodia {
13
14         @Id
15         @GeneratedValue(strategy = GenerationType.IDENTITY)
16         private long id;
17         private String melodia;
18         private String intervalos;
19         private String instrumento;
20         private String escala;
21         private Integer oitavas;
22         private Integer bpm;
23         private LocalDateTime dataGeracao;
24
25     }
```

Fonte: Autoria própria(2025).

A classe GeracaoController é um controlador responsável por lidar com as requisições HTTP relacionadas à geração e recuperação de melodias no sistema. Ela é anotada com @RestController, indicando que expõe endpoints acessíveis via HTTP, e com @RequestMapping("/geracao"), definindo a rota base para suas requisições.O controlador utiliza a anotação @Autowired para injetar automaticamente o serviço MelodiaService, responsável pela lógica de geração e recuperação das melodias armazenadas no banco de dados. Essa injeção permite que o controlador delegue a manipulação dos dados ao serviço sem a necessidade de instanciá-lo manualmente.

O primeiro ponto de acesso, definido pelo método generateMelodia, utiliza a anotação @PostMapping para receber uma requisição do tipo POST contendo um objeto ParametrosDTO no corpo da requisição. Esse objeto é passado para o serviço melodiaService, que gera uma nova entidade Melodia e a salva no banco de dados. O método retorna um Respon-

seEntity<Long> contendo o ID da melodia recém-criada, dentro de uma entidade que indica o sucesso (status 200) da requisição.

O segundo ponto de acesso, getMidiMelodia, mapeado com @GetMapping("/id"), permite recuperar a melodia correspondente ao ID informado na URL. Utilizando o serviço melodiaService, ele busca a melodia no banco de dados e, caso encontrada, cria um arquivo MIDI temporário (File.createTempFile). Em seguida, usa a classe MidiFileManager da biblioteca JFugue para converter a sequência da melodia (Pattern) em um arquivo MIDI e lê seus bytes com Files.readAllBytes. Por fim, encapsula os dados no objeto de transferência de dados ResponseMelodiaMidiDTO, retornando-os em um ResponseEntity. Na Listagem 10, apresenta-se a implementação dessa classe.

Listagem 10 – Classe GeracaoController

```
1  package GeradorMelodias.GeradorMelodias.controller;
2  import GeradorMelodias.GeradorMelodias.dto.generic.ParametrosDTO;
3  import GeradorMelodias.GeradorMelodias.dto.response.ResponseMelodiaMidiDTO;
4  import GeradorMelodias.GeradorMelodias.entity.Melodia;
5  import GeradorMelodias.GeradorMelodias.service.MelodiaService;
6  import org.jfugue.midi.MidiFileManager;
7  import org.jfugue.pattern.Pattern;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.*;
11 import java.io.File;
12 import java.nio.file.Files;
13 import java.util.Optional;
14
15 @RestController
16 @RequestMapping("/geracao")
17 public class GeracaoController {
18
19     @Autowired
20     private MelodiaService melodiaService;
21
22     @PostMapping
23     public ResponseEntity<Long> generateMelodia
24     (@RequestBody ParametrosDTO data) throws Exception {
25         Melodia melodia = melodiaService.gerarMelodia(data);
26
27         return ResponseEntity.ok().body(melodia.getId());
28     }
29
30     @GetMapping("/{id}")
31     public ResponseEntity<ResponseMelodiaMidiDTO>
32     getMidiMelodia(@PathVariable long id) throws Exception {
33         Optional<Melodia> melodia = melodiaService.findById(id);
34         File midiFile = File.createTempFile("melodiaTemp" + id ,".mid");
35         System.out.println(midiFile.toPath());
36         MidiFileManager.savePatternToMidi
37         (new Pattern(melodia.get().getMelodia()), midiFile);
38         byte[] midiBytes = Files.readAllBytes(midiFile.toPath());
39         ResponseMelodiaMidiDTO response = new ResponseMelodiaMidiDTO(
40                 melodia.get().getId(),
41                 midiBytes
42         );
43         return ResponseEntity.ok().body(response);
44     }
45
46
47 }
48
49
50
```

Fonte: Autoria própria(2025).

A classe Avaliacao representa a entidade responsável por armazenar as avaliações das melodias no sistema. Ela utiliza a anotação @Entity, que indica seu mapeamento para uma tabela no banco de dados, e a anotação @Table(name = "avaliacao"), definindo explicitamente o nome da tabela correspondente.Cada avaliação possui um identificador único (id), cujo valor é gerado automaticamente pelo banco de dados devido à anotação @GeneratedValue(strategy = GenerationType.IDENTITY). Além disso, a classe contém um campo nota, representado por um número inteiro, e um campo sensacao, que é um enumerador (Sensacao), mapeado por meio da anotação @Enumerated(EnumType.STRING), garantindo que o valor seja armazenado como texto no banco de dados.A avaliação também está associada a uma melodia específica, utilizando um relacionamento um para um mapeado usando a anotação @OneToOne. A anotação @JoinColumn(name = "id_melodia", nullable = false) define a chave estrangeira que referencia a entidade Melodia, garantindo que toda avaliação esteja vinculada a uma melodia existente. Na Listagem 11, apresenta-se a implementação dessa classe.

Listagem 11 – Classe Avaliação

```
1  package GeradorMelodias.GeradorMelodias.entity;
2  import GeradorMelodias.GeradorMelodias.enums.Sensacao;
3  import jakarta.persistence.*;
4  import lombok.Getter;
5  import lombok.Setter;
6  @Table(name = "avaliacao")
7  @Entity
8  @Getter
9  @Setter
10 public class Avaliacao {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private long id;
14
15     private Integer nota;
16
17     @Enumerated(EnumType.STRING)
18     private Sensacao sensacao;
19
20     @OneToOne
21     @JoinColumn(name = "id_melodia", nullable = false)
22     private Melodia melodia;
23 }
24
25
```

Fonte: Autoria própria(2025).

A classe AvaliacaoController é responsável por lidar com as requisições HTTP relacionadas à avaliação das melodias geradas no sistema. Essa classe é marcada com a anotação @RestController, indicando que fornece endpoints acessíveis via HTTP, e com @RequestMapping("/avaliacao"), definindo a rota base para suas requisições. O controlador utiliza a anotação

@Autowired para injetar automaticamente o serviço AvaliacaoService, que contém a lógica de processamento e persistência das avaliações no banco de dados.

O único ponto de acesso presente é definido pelo método receiveAvaliacao, que utiliza a anotação @PostMapping para receber requisições do tipo POST contendo um objeto de transferência de dados CreateAvaliacaoDTO no corpo da requisição, representado pelo parâmetro data com a anotação @RequestBody. Esse objeto contém os dados necessários para registrar uma avaliação.Ao ser chamado, o método encaminha os dados para o serviço AvaliacaoService, que realiza a operação de armazenamento da avaliação. Caso a operação seja concluída com sucesso, o método retorna uma resposta HTTP de sucesso (ResponseEntity.ok().build()), sem necessidade de um corpo adicional. Na Listagem 12, apresenta-se a implementação dessa classe e seu método.

Listagem 12 – Classe Avaliação Controller

```java
package GeradorMelodias.GeradorMelodias.controller;

import GeradorMelodias.GeradorMelodias.dto.request.CreateAvaliacaoDTO;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import GeradorMelodias.GeradorMelodias.service.AvaliacaoService;

@RestController
@RequestMapping("/avaliacao")
public class AvaliacaoController {

    @Autowired
    private AvaliacaoService avaliacaoService;



    @PostMapping
    public ResponseEntity<?> receiveAvaliacao
    (@RequestBody CreateAvaliacaoDTO data)
    throws Exception {
        avaliacaoService.saveAvaliacao(data);

        return ResponseEntity.ok().build();
    }

}

```

Fonte: Autoria própria(2025).

A classe MelodiaRepository é um repositório responsável por intermediar a comunicação entre o banco de dados e o sistema, utilizando a interface JpaRepository para fornecer operações básicas de persistência para a entidade Melodia. Ela é anotada com @Repository, indicando que sua implementação será gerenciada automaticamente pelo Spring e injetada

onde necessário. Além dos métodos padrão fornecidos pelo JpaRepository, esta classe define consultas personalizadas utilizando a anotação @Query, permitindo a obtenção de informações mais complexas a partir dos dados armazenados.

O primeiro método, countByParametroAndSensacao, recebe um nome de parâmetro (BPM, Escala, Instrumento ou Oitavas) e uma Sensacao, que representa a sensação utilizada para avaliações de melodias geradas que será utilizada na análise. A consulta realiza uma contagem das avaliações associadas a melodias filtradas pela sensação informada, agrupando os resultados de acordo com o parâmetro escolhido. No caso do BPM, os valores são agrupados em intervalos de dez unidades para facilitar a análise. O retorno é uma lista de vetores de objetos contendo o valor do parâmetro escolhido, a sensação associada e a quantidade de ocorrências encontradas.

O segundo método, countByParametroAndNota, funciona de maneira semelhante, mas realiza a contagem com base na nota atribuída pelos avaliadores. Ele recebe um parâmetro de categorização e um intervalo de notas (minNota e maxNota). A consulta agrupa os resultados de acordo com o valor do parâmetro informado e contabiliza as ocorrências para cada nota dentro do intervalo especificado. Caso o parâmetro escolhido seja Sensacao, o agrupamento considera diretamente essa informação ao invés das características musicais.

Na Listagem 13, é demonstrada a implementação desta classe, que contém a relação direta entre o banco de dados e o funcionamento interno da aplicação, consolidando as informações das avaliações para fins de análise.

Listagem 13 – Classe MelodiaRepository

```
1  package GeradorMelodias.GeradorMelodias.repository;
2  import GeradorMelodias.GeradorMelodias.entity.Melodia;
3  import GeradorMelodias.GeradorMelodias.enums.Sensacao;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.data.jpa.repository.Query;
6  import org.springframework.data.repository.query.Param;
7  import org.springframework.stereotype.Repository;
8  import java.util.List;
9
10 @Repository
11 public interface MelodiaRepository extends JpaRepository<Melodia, Long> {
12     @Query("SELECT " +
13             "CASE " +
14             "  WHEN :parametro = 'BPM'
15             THEN CONCAT(FLOOR(m.bpm / 10) * 10, '-',
16             FLOOR(m.bpm / 10) * 10 + 10) " +
17             "  WHEN :parametro = 'Escala' THEN m.escala " +
18             "  WHEN :parametro = 'Instrumento' THEN m.instrumento " +
19             "  WHEN :parametro = 'Oitavas' THEN m.oitavas " +
20             "  ELSE 'Desconhecido' " +
21             "END AS parametroValor, " +
22             "a.sensacao, COUNT(a) AS quantidade " +
23             "FROM Melodia m " +
24             "JOIN Avaliacao a ON m.id = a.melodia.id " +
25             "WHERE a.sensacao = :sensacao " +
26             "GROUP BY parametroValor, a.sensacao ORDER BY quantidade desc")
27     List<Object[]> countByParametroAndSensacao(
28     @Param("parametro") String parametro,
29     @Param("sensacao") Sensacao sensacao);
30
31     @Query("SELECT " +
32             "CASE " +
33             "WHEN :parametro = 'BPM' THEN
34             CONCAT(FLOOR(m.bpm / 10) * 10, '-',
35             FLOOR(m.bpm / 10) * 10 + 10)" +
36             "  WHEN :parametro = 'Escala' THEN m.escala " +
37             "  WHEN :parametro = 'Instrumento' THEN m.instrumento " +
38             "  WHEN :parametro = 'Oitavas' THEN m.oitavas " +
39             "  WHEN :parametro = 'Sensacao' THEN a.sensacao " +
40             "  ELSE 'Desconhecido' " +
41             "END AS parametroValor, " +
42             "a.nota, COUNT(a) AS quantidade " +
43             "FROM Melodia m " +
44             "JOIN Avaliacao a ON m.id = a.melodia.id " +
45             "WHERE a.nota BETWEEN :minNota AND :maxNota " +
46             "GROUP BY parametroValor, a.nota ORDER BY quantidade desc")
47     List<Object[]> countByParametroAndNota(@Param("parametro")
48     String parametro, @Param("minNota") Integer minNota,
49     @Param("maxNota") Integer maxNota);
50 }
51
```

Fonte: Autoria própria(2025).

A classe AnaliseController é um controlador responsável por lidar com as requisições HTTP que retornam informações para análise a serem apresentadas no front-end. Ela é anotada com @RestController, o que indica que ela expõe endpoints acessíveis via HTTP, e com @RequestMapping("/analises"), definindo a rota base para suas requisições. O controlador utiliza a anotação @Autowired para injetar automaticamente o serviço MelodiaService, que é responsável pela lógica de análise e avaliação das informações armazenadas no sistema.

O primeiro ponto de acesso, mapeado com @GetMapping("/sensacao-parametro"), define um uma requisição do tipo GET. Ele recebe um objeto de transferência de dados RequestAnaliseParametroSensacaoDTO no corpo da requisição, que contém os parâmetros necessários para realizar a análise com base em sensação e parâmetro. O método chama o serviço MelodiaService, que realiza a análise e retorna uma lista de objetos AnaliseSensacaoAvaliadaPorParametro.

O segundo ponto de acesso, mapeado com @GetMapping("/nota-parametro"), mapeia uma requisição GET que recebe um objeto de transferência de dados RequestAnaliseParametroNotaDTO no corpo da requisição. Esse objeto contém os parâmetros necessários para realizar a análise com base em nota e parâmetro. O método chama o serviço MelodiaService, que realiza a análise e retorna uma lista de objetos AnaliseNotaAvaliadaPorParametro. Na Listagem 14, é demonstrado o código dessa classe, que organiza e direciona as requisições relacionadas à análise para os métodos do serviço correspondente.

Listagem 14 – Classe AnaliseController

```
1  package GeradorMelodias.GeradorMelodias.controller;
2  import GeradorMelodias.GeradorMelodias.dto.
3  request.RequestAnaliseParametroNotaDTO;
4  import GeradorMelodias.GeradorMelodias.dto.
5  request.RequestAnaliseParametroSensacaoDTO;
6  import GeradorMelodias.GeradorMelodias.dto.
7  response.AnaliseNotaAvaliadaPorParametro;
8  import GeradorMelodias.GeradorMelodias.dto.
9  response.AnaliseSensacaoAvaliadaPorParametro;
10 import GeradorMelodias.GeradorMelodias.
11 service.MelodiaService;
12 import org.springframework.beans.factory.annotation.Autowired;
13 import org.springframework.web.bind.annotation.*;
14
15 import java.util.List;
16
17 @RestController
18 @RequestMapping("/analises")
19 public class AnaliseController {
20
21     @Autowired
22     private MelodiaService melodiaService;
23
24     @GetMapping("/sensacao-parametro")
25     public List<AnaliseSensacaoAvaliadaPorParametro>
26     getAnalisePorSensacaoEParametro
27     (RequestAnaliseParametroSensacaoDTO dto) {
28         return melodiaService.getAnaliseSensacaoPorParametro(dto);
29     }
30
31     @GetMapping("/nota-parametro")
32     public List<AnaliseNotaAvaliadaPorParametro> getAnalisePorNotaEParametro
33     (RequestAnaliseParametroNotaDTO dto) {
34         return melodiaService.getAnaliseSensacaoPorParametro(dto);
35     }
36 }
37
38
39
40
```

Fonte: Autoria própria(2025).

4.5 Comentário sobre funcionamento do sistema

O sistema de geração de melodias foi utilizado brevemente por um número reduzido de usuários, e, de maneira geral, as melodias geradas foram avaliadas agradáveis e concisas dentro dos parâmetros estabelecidos pela teoria musical, como o uso de instrumentos, bpm,

variação de oitavas e escala. No entanto, foi apontado que o sistema ainda apresentava um caráter artificial, especialmente no que diz respeito à classificação das sensações.

Apesar da adequação das melodias do ponto de vista técnico e da concisão com a teoria musical, a conexão emocional com as criações mostrou-se limitada. Em muitos casos, as melodias geradas não despertaram sensações tão fortes ou autênticas, sugerindo que a ausência da influência humana no processo de execução poderia ter contribuído para essa falta de expressividade. O fato de o sistema utilizar uma interpretação não humana ao tocar as melodias parece ter sido um fator que reforçou a característica sintética das composições, impedindo uma maior imersão emocional.

Para aumentar a expressividade e tornar as melodias mais envolventes, seria interessante introduzir variações dinâmicas ao longo da execução. Alterações nas intensidades, criando crescendos e decrescendo naturais, bem como variações no tempo e no toque das notas, poderiam simular uma interpretação mais humana. Isso permitiria que, por exemplo, uma parte mais intensa da melodia fosse seguida por uma passagem suave, criando um efeito de "respiração"musical que conecta melhor com o ouvinte. Além disso, o uso de técnicas de articulação, como staccato e legato, ajudaria a adicionar uma camada de expressividade ao som, assim como a manipulação do timbre, utilizando variações na intensidade do ataque e efeitos como o vibrato, poderia dar um toque mais orgânico à execução, imitando variações presentes em performances reais.

Outra possibilidade seria incorporar uma imitação da interpretação humana, especialmente no que diz respeito ao tempo flexível. A variação sutil no ritmo, como o rubato, poderia ser usada para proporcionar uma sensação de liberdade e fluidez, algo que é típico nas execuções humanas e que poderia melhorar a percepção emocional da melodia.

Em suma, embora o sistema tenha apresentado um bom desempenho em termos de geração técnica e teórica das melodias, há um claro espaço para aprimorar a forma como elas são geradas. A incorporação de características mais expressivas, que simulem uma interpretação humana e incluam variações dinâmicas, articulação e timbre, poderia contribuir para composições mais emocionais e envolventes, permitindo que as melodias geradas não apenas sigam a teoria musical, mas também despertem uma resposta emocional mais profunda nos ouvintes.

4.6 Disponibilização do código fonte

O código-fonte está disponível no seguinte link, em um repositório github: https://github.com/VictorCFerreira/GeradorMelodias.

5 CONCLUSÃO

Este trabalho retratou a implementação de um sistema web para geração procedural, aplicando diferentes heurísticas, avaliação e análise de melodias. O *framework* Spring Boot foi essencial para o desenvolvimento do sistema, permitindo uso da linguagem java para o formato web. As IDEs Visual Studio Code e Intellij foram grandes facilitadores também para o trabalho, devido a sua interface prática. Utilizando a biblioteca JFugue, foi possível gerar melodias proceduralmente, e, seguindo determinadas heurísticas, é possível obter resultados razoavelmente agradáveis, porém, ainda há possibilidade, devido a aleatoriedade parcial da geração, de produzir gerações desagradáveis.

Dentre as dificuldades no desenvolvimento do sistema, as mais notáveis foram: uso e instalação da biblioteca JFugue, que pode ser superada com a busca em fóruns de internet e documentação, trabalho com funções de reprodução de som em react, também superada com pesquisa, porém, mais focada em fóruns e repositórios públicos.

O desenvolvimento desse sistema apresenta relevância tanto para o mercado de composição, quanto para a geração computacional, visto que a geração é feita a partir de heurísticas e parametrização, demonstrando a sua possibilidade de geração procedural, e, trazendo estatísticas analisáveis para a área musical. A possibilidade de download das melodias permite também o seu uso como auxílio em processo criativo, ou uso externo, salvando melodias agradáveis para aplicação em jogos, compartilhamento, uso do usuário em suas composições, ou mesmo uso livre próprio. Apesar do formato simples de geração utilizando heurísticas, o método de avaliação incluindo sensações oferece destaque a este trabalho no campo de análise, permitindo estudo de fatores de teoria musical que tendem a determinadas sensações.

Apesar de dificuldades de implementação na parte de som, como citado anteriormente, as telas criadas foram satisfatórias, usando as ferramentas citadas no Quadro 2, foram implementadas visualizações das informações analisadas utilizando gráficos, e também uma interface para a geração e avaliação de melodias.

É notável que a extensão das possibilidades desta proposta é maior do que as implementadas nesta pesquisa. Existe viabilidade para trabalhos futuros com recursos extras para o sistema de geração e análise criado, em particular, desenvolvimento de geração de músicas de extensão completa, utilizando múltiplas gerações em sequência. O uso de notas simultâneas na geração aplicando conceitos de harmonia, tanto para gerações de melodias curtas como complementando a possibilidade de gerações de extensão completa oferece a possibilidade de gerações que se aproximam melhor a composições humanas. Compartilhamento de melodias geradas em redes sociais de forma direta utilizando integração, oferecendo dentro do próprio sistema uma possibilidade de divulgar seus resultados. Por fim, uso de um algoritmo de *machine learning* para consumir informações acerca das análises geradas, criando um sistema adaptativo a seus próprios resultados, que melhora os padrões de geração conforme acumula informações de uso.

REFERÊNCIAS

Academia Musical. **Conheça a duração das notas musicais**. 2023. Disponível em: https://www.academiamusical.com.pt/tutoriais/a-duracao-das-notas-musicais/. Acesso em: 29 jan. 2025.

ADELE. **Someone Like You**. [*S.l.*]: XL Recordings, 2011. Disponível em: https://www.discogs.com/Adele-Someone-Like-You/master/1071274. Acesso em: 29 jan. 2025.

AGNOL, J. L. H. D. **Geração Procedural de Melodias por Gênero Usando Aprendizado de Máquina**. dezembro 2022. Trabalho de Conclusão de Curso — Universidade Tecnológica Federal do Paraná (UTFPR), Paraná, Brasil, dezembro 2022.

ALMEIDA, J.; SANTOS, M. A interação entre melodia e harmonia na composição musical. **Revista Brasileira de Música**, v. 10, n. 2, p. 45–60, 2022.

BARROS, C. K.; ANSAY, N. N. A experiência da composição musical na musicoterapia: revisão de literatura. **Revista Brasileira de Musicoterapia**, 2015. Disponível em: https://musicoterapia.revistademusicoterapia.mus.br/index.php/rbmt/article/download/93/74/192.

COMPOSERS, A. American Society of; ASCAP, P. **Twinkle, Twinkle, Little Star**. 2003. Disponível em: https://essentialmusicpublishing.com/songs/twinkle-twinkle-little-star/. Acesso em: 29 jan. 2025.

COSTA, P.; PEREIRA, A. A influência da tecnologia na criação melódica. *In*: **Anais do Congresso Brasileiro de Música**. [*S.l.*: *s.n.*], 2023. v. 15, p. 123–135.

Descomplicando a Música. **Compasso musical**. 2014. Disponível em: https://www.descomplicandoamusica.com/compasso-musical. Acesso em: 21 fev. 2025.

DOBBERT, F. **Modulação e Transformações Musicais em Beethoven**. 2020. 45–60 p. Disponível em: https://www.revistaestudosmusicais.com/fritzdobbert2020. Acesso em: 29 jan. 2025.

Araújo, E. R. **Análise da influência do TikTok na indústria musical através da engenharia de avaliações**. 2020. Disponível em: https://repositorio.ufpb.br/jspui/bitstream/123456789/17724/1/TCC%20EMILYN%20ROQUE%20ARA%C3%9AJO.pdf.

Briot, J. P.; Hadjeres, G.; Pachet, F. D. **Deep Learning Techniques for Music Generation**. [*S.l.*]: Springer, 2020.

Britannica, T. E. o. E. **melody**. 2024. Disponível em: https://www.britannica.com/art/melody. Acesso em 25 de março de 2024.

Dowling, W. J.; Harwood, D. L. **Music Cognition**. [*S.l.*]: Academic Press, 1986.

Ferreira, L.; Costa, M. Análise e síntese de Áudio com inteligência artificial. *In*: **Anais do Simpósio Brasileiro de Computação Musical**. [*S.l.*: *s.n.*], 2023.

Fávero, L. P. L. *et al.* **Análise de Dados: Modelagem Multivariada para Tomada de Decisões**. Rio de Janeiro: Elsevier, 2009.

Gobetti, I. A. **Sistema Baseado em Heurísticas para Geração Procedural de Melodias**. dezembro 2022. Trabalho de Conclusão de Curso de Graduação — Universidade Tecnológica Federal do Paraná, Paraná, Brasil, dezembro 2022.

Hair, J. F. *et al.* **Multivariate Data Analysis**. [*S.l.*]: Pearson, 2014.

Kahneman, D.; Tversky, A. Judgment under uncertainty: Heuristics and biases. **Science**, v. 185, n. 4157, p. 1124–1131, 1974. Disponível em: https://www.science.org/doi/10.1126/science.185.4157.1124.

Moraes, J. D. d.; Tabak, B. M. As heurísticas e vieses da decisão judicial: análise econômico-comportamental do direito. **Revista Direito GV**, v. 14, n. 2, p. 321–340, 2018. Disponível em: https://periodicos.fgv.br/revdireitogv/article/download/77113/73919/160316.

Nunes, A. L.; Santos, F. P. Cognição social e heurísticas: A influência dos estereótipos na tomada de decisão. **Psicologia para América Latina**, n. 36, p. 45–58, 2021. Disponível em: http://pepsic.bvsalud.org/scielo.php?script=sci_arttext&pid=S1870-350X2021000200008.

Pacheco, J. *et al.* Algoritmos procedurais na produção musical. **Revista Brasileira de Computação Aplicada**, v. 13, n. 2, 2021.

Pereira, A. A.; Madruga, Z. E. de F. Música e modelagem matemática: Representação de uma escala musical por meio de modelo matemático. **Revista de Educação Matemática**, v. 7, n. 1, p. 1–33, 2020. Disponível em: https://revistas.pucsp.br/emd/article/download/45029/pdf_1.

Schmidt, R. **Intervalo Musical - conceito e classificação**. 2018. www.encorda.com.br/blod/intervalo-musical.

Silva, F.; Oliveira, R. A importância da melodia na construção da identidade musical. **Revista de Música e Cultura**, v. 12, n. 3, p. 34–50, 2019.

Silva, J. P. **Intervalos musicais: semitons e tons no piano**. 2025. Disponível em: https://avancandonamusica.com.br/o-que-e-um-semitom-e-um-tom.

Silva, M. C.; Almeida, J. P. A aplicação de heurísticas na tomada de decisão em saúde. **Revista Brasileira de Saúde**, v. 10, n. 1, p. 45–60, 2020. Disponível em: https://www.scielo.br/j/rbs/a/1234567890/.

Silva, P.; Oliveira, A. Algoritmos evolutivos na composição musical. *In*: **Anais do Congresso Brasileiro de Inteligência Computacional**. [*S.l.*: *s.n.*], 2022.

Sodré, D. F. de P. A matemática da escala musical. **Revista de Matemática Aplicada**, v. 15, n. 2, p. 45–60, 2015.

Souza, R.; Almeida, B. Redes neurais na geração de música. **Revista de Inteligência Artificial Aplicada**, v. 15, n. 1, 2023. Disponível em: link.

Titon, J. T. **Worlds of Music: An Introduction to the Music of the World's Peoples**. [*S.l.*]: Cengage Learning, 2014.

Tonetto, L. M. *et al.* O papel das heurísticas no julgamento e na tomada de decisão sob incerteza. **Estudos de Psicologia (Campinas)**, v. 23, n. 2, p. 187–195, 2006. Disponível em: https://www.scielo.br/j/estpsi/a/RnbtvVjsY9wgn7FLpxcGGHh/.

FERRAZ, S. **Composição musical como campo de diálogo: para além das disciplinas**. 2013. Departamento de Música – Universidade de São Paulo – USP. Disponível em: https://www.eca.usp.br/acervo/producao-academica/002721419.pdf.

JACKSON, M. **Billie Jean**. Epic Records, 1982. Disponível em: https://www.discogs.com/Michael-Jackson-Billie-Jean/master/58731.

JEREMY, R. **Teoria Musical: Uma Introdução**. São Paulo: Editora Musica, 2021.

JONES, S. **Rock Formation: Music, Technology, and Mass Communication**. Newbury Park: Sage Publications, 1992.

LINDENMAYER, A. Mathematical models for cellular interactions in development i. filaments with one-sided inputs. **Journal of Theoretical Biology**, v. 18, n. 3, p. 280–299, 1968.

MANDELBROT, B. B. **The fractal geometry of nature**. W. H. Freeman and Company, 1982. Disponível em: https://www.freeman.com/fractal-geometry.

MANOVICH, L. **The Language of New Media**. Cambridge: MIT Press, 2001.

MARTINS, M. **Ritmo na Música**. 2018. Disponível em: https://knoow.net/arteseletras/musica/ritmo/. Acesso em: 27 jan. 2025.

Maxis. **Spore**. 2008. Disponível na Steam: https://store.steampowered.com/app/17390/Spore/. Acesso em: 27 jan. 2025.

MCMILLEN, F. H. E. **The Binding of Isaac**. 2011. Disponível na Steam: https://store.steampowered.com/app/113200/The_Binding_of_Isaac/. Acesso em: 27 jan. 2025.

Mojang Studios. **Minecraft**. 2011. Disponível em: https://www.minecraft.net/. Acesso em: 2 fev. 2025.

MORAES, A. da S. Física na música. **GPET física Unicentro**, 2015.

NOVACK, Z. *et al.* Unsupervised lead sheet generation via semantic compression. **arXiv preprint**, 2011. ArXiv:2310.10772.

Opus Ensino Musical. **O que são tom, semitom, sustenido e bemol?** 2018. Disponível em: aprendamusica.opus3ensinomusical.com.br/o-que-e-tom-semitom-sustenido-e-bemol/. Disponível em: Acesso em: 27 jan. 2025.

PERLIN, K. An image synthesizer. **ACM SIGGRAPH Computer Graphics**, ACM, v. 19, n. 3, p. 287–296, 1985. Disponível em: https://dl.acm.org/doi/10.1145/325165.325247.

PISTON, W. **Teoria Musical: Um Curso Completo**. 7. ed. [*S.l.*]: Editora XYZ, 1962.

PISTON, W.; DEVOTO, M. **Harmony**. [*S.l.*]: W.W. Norton & Company, 1987.

SCHOENBERG, A. **Teoria da Música**. 1ª. ed. São Paulo: Editora Perspectiva, 2011.

SILVA, M. V. da; MARSON, F.; CASSOL, V. **gameBITS Framework: A procedural content generation framework for digital games**. 2016.

SMITH, G.; CHA, M.; WHITEHEAD, J. A framework for analysis of 2d platformer levels. *In*: **Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games**. [*S.l.*: *s.n.*], 2010. p. 351–358.

SOUZA, M. Retenção e reconhecimento de melodias: um estudo empírico. **Revista de Psicologia da Música**, v. 5, n. 2, p. 101–115, 2020.

TAYLOR, J.; MOZART, W. A. **Twinkle, Twinkle, Little Star**. 1806. Música popular infantil. Versão melódica de Wolfgang Amadeus Mozart.

TOGELIUS, J. *et al.* Search-based procedural content generation: A taxonomy and survey. **IEEE Transactions on Computational Intelligence and AI in Games**, v. 3, n. 3, p. 172–186, 2011.

YANNAKAKIS, G. N.; TOGELIUS, J. **Artificial Intelligence and Games**. New York: Springer, 2018.